

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC - KỸ THUẬT MÁY TÍNH



BÁO CÁO THỰC TẬP

Đề tài

Recommendation System

GVHD: Nguyễn Văn Triệu
SV: Phan Thảo Vy - 2252930

TP. HỒ CHÍ MINH, THÁNG 10/2024

Contents

1	Tổng quan lý thuyết về recommendation system	4
1.1	Khái quát chung	4
1.2	Collaborative Filtering Models	4
1.2.1	Neighborhood-Based Collaborative Filtering (memory-based algorithms)	5
1.2.2	Model-Based Collaborative Filtering	8
1.3	Content-based Recommender Systems	9
1.3.1	Đặc điểm chính	9
1.3.2	Các thành phần cơ bản của Content-Based Systems	9
1.4	Knowledge-based recommendation system	11
2	GCN	11
2.1	Giới thiệu	11
2.2	Cách hoạt động	11
2.3	Ưu điểm	14
2.4	Nhược điểm	14
3	LightGCN	15
3.1	Giới thiệu	15
3.2	Kiến trúc mô hình	18
3.3	Light Graph Convolution (LGC) trong LightGCN	19
3.3.1	Công thức lan truyền (propagation rule)	19
3.3.2	Layer Combination (Kết hợp nhiều lớp embedding)	20
3.3.3	Dự đoán của mô hình LightGCN (Model Prediction)	21
3.4	Dạng ma trận LightGCN	21
3.4.1	Ma trận tương tác user-item (R)	21
3.4.2	Ma trận kề của đồ thị user-item (A)	21
3.4.3	Embedding khởi tạo ($E^{(0)}$)	22
3.4.4	Công thức lan truyền embedding qua các layer	22
3.4.5	Kết hợp nhiều layer để ra embedding cuối cùng	22
3.5	Huấn luyện mô hình	22
3.6	So sánh hiệu năng: LightGCN và NGCF	23
3.7	So sánh hiệu năng với các mô hình tiên tiến khác	24
3.8	Tầm quan trọng của các thành phần trong LightGCN	25
3.8.1	Layer Combination	25



3.8.2	Tác động của Symmetric Sqrt Normalization trong LightGCN	26
3.8.3	Phân tích độ mượt (Embedding Smoothness) trong LightGCN	27
3.9	Hyper-parameter	28
3.10	Tóm lại	29
3.11	Ưu điểm và nhược điểm	29
3.11.1	Ưu điểm	29
3.11.2	Nhược điểm	30
4	Giới thiệu về dataset	30
5	Hiện thực LightGCN	31
5.1	Load dữ liệu	31
5.2	Kiểm tra dữ liệu	32
5.2.1	Kích thước dữ liệu	32
5.2.2	Kiểm tra dữ liệu null	32
5.2.3	Xử lý các dữ liệu trùng lặp	33
5.2.4	Tần suất của các giá trị trong cột event	33
5.2.5	Kiểm tra độ dày của dữ liệu	35
5.2.6	Đồ thị thể hiện sự phân phối	36
5.2.7	Thống kê mô tả	40
5.3	Tiền xử lý dữ liệu	42
5.4	Hiện thực LightGCN thuần	48
5.4.1	Chuẩn bị dữ liệu	48
5.4.2	Chia train/test/validation	49
5.4.3	Tạo edge index và edge weight cho train	51
5.4.4	LightGCN	52
5.4.5	Hàm Loss	56
5.4.6	Hàm sample batch	58
5.4.7	Huấn luyện mô hình	60
5.4.8	Hàm đánh giá LightGCN	64
5.4.9	Hàm recommend	66
5.5	Hiện thực LightGCN kết hợp thêm feature	67
5.5.1	Tiền xử lý dữ liệu <i>item_properties</i>	67
5.5.2	Chuẩn bị dữ liệu đầu vào	73
5.5.3	LightGCN kết hợp feature	74
5.6	Thử nghiệm	76



6	Tổng kết	78
7	Tài liệu tham khảo	79

1 Tổng quan lý thuyết về recommendation system

1.1 Khái quát chung

- Định nghĩa: Recommendation system (hệ thống gợi ý) là một loại phần mềm hoặc thuật toán giúp đề xuất các mục nội dung phù hợp cho người dùng dựa trên dữ liệu về họ (hành vi, sở thích, thông tin cá nhân...).
- Các loại Recommendation System phổ biến:
 - Content-based Recommendation: Đề xuất dựa vào sự tương đồng giữa các mục mà người dùng đã thích trước đó (dựa vào mô tả, đặc điểm...).
 - Collaborative Recommendation: Đề xuất dựa vào hành vi của nhiều người dùng. Có hai kiểu: User-based, Item-based.
 - Knowledge-Based Recommendation: Đề xuất dựa vào các yêu cầu cụ thể, ràng buộc, luật do người dùng hoặc hệ thống quy định.
 - Hybrid Systems: Kết hợp nhiều phương pháp trên để tăng độ chính xác

1.2 Collaborative Filtering Models

- Định nghĩa: Là phương pháp gợi ý sử dụng sức mạnh tập thể của các đánh giá (ratings) từ nhiều người dùng khác nhau để dự đoán sở thích cho từng người dùng cá nhân.
- Các loại tương quan:
 - Inter-user correlation: So sánh sự giống nhau giữa các người dùng từ đó gợi ý dựa trên những người dùng giống bạn.
 - Inter-item correlation: So sánh sự giống nhau giữa các mục tiêu/sản phẩm từ đó gợi ý các sản phẩm/phim tương tự những gì bạn đã thích.
 - Phương pháp:
 - * Memory-based methods (neighborhood based collaborative filtering algorithms): Dự đoán rating của một người dùng cho một

sản phẩm dựa vào các “hàng xóm” (neighborhood) – tức là dựa vào những người dùng hoặc sản phẩm tương tự. Có hai loại chính: User-based Collaborative Filtering và Item-based collaborative filtering. Memory-based methods phù hợp khi dữ liệu rating không quá thưa, dễ triển khai, giải thích rõ ràng cho người dùng. Nhưng nếu số lượng rating quá ít, hoặc số lượng người dùng/sản phẩm cực lớn, nên chuyển sang các phương pháp model-based.

- * Model-based Methods: Là nhóm phương pháp sử dụng các kỹ thuật machine learning và data mining để xây dựng mô hình dự đoán từ dữ liệu đánh giá của người dùng.

Tóm lại: Memory-based và model-based không phải là hai thế giới tách biệt hoàn toàn. Có nhiều điểm giao thoa và thậm chí có thể chuyển đổi qua lại bằng kỹ thuật phù hợp.

1.2.1 Neighborhood-Based Collaborative Filtering (memory-based algorithms)

Neighborhood-Based Collaborative Filtering có hai phương pháp chính:

User-based collaborative filtering: Dự đoán điểm số/rating cho người dùng A dựa trên đánh giá của những người dùng “giống A” nhất (dựa trên độ tương đồng về hành vi).

- Tìm “hàng xóm” của người dùng A (dùng hàm đo độ tương đồng giữa các hàng của ma trận rating).
- Dùng trung bình có trọng số của đánh giá từ các hàng xóm này để dự đoán cho A.

Item-based collaborative filtering: Dự đoán điểm số/rating cho mục tiêu (item) B dựa trên những item tương tự B mà người dùng A đã đánh giá.

- Tìm các mục tiêu giống B (dùng hàm đo độ tương đồng giữa các cột của ma trận rating).
- Dùng trung bình có trọng số từ các rating của A trên các item này để dự đoán cho B.

Hai dạng toán chính của bài toán recommendation system:

1. Dự đoán giá trị rating: Dự đoán xem một người dùng (user u) sẽ đánh giá (rating) thế nào cho một sản phẩm cụ thể (item j) – tức là điền giá trị vào ô r_{uj} còn thiếu.
2. Gợi ý top-k sản phẩm hoặc top-k người dùng: Xác định k sản phẩm phù hợp nhất để gợi ý cho một người dùng (hoặc ngược lại: k người dùng phù hợp nhất cho một sản phẩm).

Cả hai bài toán có mối liên hệ với nhau. Để giải quyết top-k recommendation, thông thường phải dự đoán rating cho tất cả các sản phẩm chưa có đánh giá, từ đó chọn ra k sản phẩm có điểm dự đoán cao nhất. Tương tự, để tìm top-k người dùng phù hợp với một sản phẩm, hệ thống dự đoán rating của sản phẩm đó cho từng người dùng, sau đó lấy top-k người dùng điểm cao nhất.

Ý tưởng cơ bản của các phương pháp dựa vào láng giềng (neighborhood-based) là sử dụng độ tương đồng giữa người dùng với người dùng (user-user similarity) hoặc giữa sản phẩm với sản phẩm (item-item similarity) để đưa ra gợi ý từ ma trận đánh giá. Có hai nguyên tắc cơ bản trong các mô hình này:

- User-based models: Những người dùng giống nhau sẽ có đánh giá giống nhau trên cùng một vật. User-based neighborhoods được xác định nhằm tìm ra những người dùng tương tự với người dùng mục tiêu mà ta muốn dự đoán rating. Để xác định nhóm láng giềng cho người dùng mục tiêu i (người dùng mà ta cần dự đoán rating), ta cần tính độ tương đồng giữa người dùng này với tất cả các người dùng còn lại. Vì vậy, cần phải xây dựng một hàm đo độ tương đồng giữa các điểm đánh giá mà người dùng đã cho. Một chỉ số thường dùng để đo độ tương đồng $\text{Sim}(u, v)$ giữa vector rating của hai user u và v là hệ số tương quan Pearson.

$$\text{RawCosine}(u, v) = \frac{\sum_{k \in I_u \cap I_v} r_{uk} \cdot r_{vk}}{\sqrt{\sum_{k \in I_u} r_{uk}^2} \cdot \sqrt{\sum_{k \in I_v} r_{vk}^2}}$$

Raw Cosine là một cách tính độ tương đồng giữa hai người dùng (hoặc hai sản phẩm) dựa trên vector các điểm rating thực tế mà họ đã chấm, không cần chuẩn hóa (trừ trung bình) như Pearson correlation. Hay nói cách khác Raw Cosine để đo xem hai user có xu hướng “chấm điểm cùng hướng” hay không, bất kể mỗi người thích chấm cao hay thấp. Raw Cosine không loại bỏ sự khác biệt về mức độ (scale). Nếu một user lúc nào cũng chấm cao hơn người khác một chút, Cosine similarity vẫn

ra giá trị cao. Trong khi đó Pearson correlation loại bỏ sự khác biệt về mức độ bằng cách trừ trung bình từng user, chỉ so sánh “xu hướng tăng giảm”. Raw Cosine dùng khi không cần hiệu chỉnh thiên lệch cá nhân.

$$\text{RawCosine}(u, v) = \frac{\sum_{k \in I_u \cap I_v} r_{uk} \cdot r_{vk}}{\sqrt{\sum_{k \in I_u} r_{uk}^2} \cdot \sqrt{\sum_{k \in I_v} r_{vk}^2}}$$

Dự đoán ratings theo hai cách:

- Dự đoán bằng rating gốc (raw ratings)
- Dự đoán bằng rating đã trừ trung bình (mean-centered ratings): Mean-centering giúp dự đoán thứ tự ưu tiên tốt hơn. Do nó giúp loại bỏ thiên lệch cá nhân. Mỗi người dùng có “khẩu vị” chấm điểm khác nhau. Có người luôn chấm điểm rất cao cho mọi sản phẩm (dù thích hay không). Nhưng lại có người lại rất khắt khe, luôn chấm thấp hơn bình thường. Nếu không mean-centering thì các user “hào phóng” hoặc “khắt khe” khi so sánh với nhau sẽ được coi là giống nhau về hướng chấm điểm (Cosine similarity vẫn cao), dù thực ra xu hướng thích/không thích lại khác nhau. Khi đã trừ đi trung bình từng người, mỗi giá trị rating thể hiện mức độ thích/không thích so với thói quen riêng của từng người. Như vậy, hai user sẽ được coi là “giống nhau” nếu họ có xu hướng thích/không thích cùng các sản phẩm, bất kể họ là người hào phóng hay khó tính.

$$\hat{r}_{ij} = \mu_u + \frac{\sum_{v \in P_{u,j}} \text{Sim}(u, v) \cdot s_{ij}}{\sum_{v \in P_{u,j}} |\text{Sim}(u, v)|} = \mu_u + \frac{\sum_{v \in P_{u,j}} \text{Sim}(u, v) \cdot (r_{ij} - \mu_v)}{\sum_{v \in P_{u,j}} \text{Sim}(u, v)}$$

Long-tail distribution: Trong dữ liệu thực tế, hầu hết các sản phẩm (ví dụ: phim) chỉ được một số ít người đánh giá, nhưng một số sản phẩm/phim rất phổ biến lại được rất nhiều người đánh giá. Các item phổ biến này (ở “phần đầu” của biểu đồ) xuất hiện rất nhiều trong lịch sử đánh giá của nhiều user.

- Item-based models: Thay vì tập trung vào việc so sánh giữa các người dùng, item-based recommendation tập trung vào mối liên hệ giữa các sản phẩm (item). Nếu một user đã thích/không thích một số item nào đó, hệ thống sẽ gợi ý các item giống nhất với những item user đã từng đánh giá cao.

Cách hoạt động:

- Xây dựng ma trận độ tương đồng giữa các item: Tính độ tương đồng (similarity) giữa từng cặp sản phẩm. Phương pháp phổ biến là Adjusted Cosine Similarity (hoặc Pearson nếu phù hợp).

$$\text{AdjustedCosine}(i, j) = \frac{\sum_{u \in U_i \cap U_j} s_{iu} \cdot s_{ju}}{\sqrt{\sum_{u \in U_i \cap U_j} s_{iu}^2} \cdot \sqrt{\sum_{u \in U_i \cap U_j} s_{ju}^2}}$$

Để tính similarity giữa item i và item j:

- * Chỉ xét các user đã đánh giá cả hai item này. Với mỗi user, lấy điểm rating đã được trừ đi điểm trung bình của user đó.
 - * Áp dụng công thức cosine cho hai vector này.
- Lấy điểm rating của user U trên các item “giống nhất” vừa chọn. Tính trung bình có trọng số các điểm rating này (trọng số là mức độ similarity với item T). Công thức:

$$\hat{r}_{ut} = \frac{\sum_{j \in Q_t(u)} \text{AdjustedCosine}(j, t) \cdot r_{uj}}{\sum_{j \in Q_t(u)} |\text{AdjustedCosine}(j, t)|}$$

Item-based thường chính xác hơn. Lý do: Item-based dùng rating của chính user trên các item tương tự để dự đoán rating cho item mới. Ví dụ: Nếu bạn từng thích các phim lịch sử A, B, C, thì khả năng bạn sẽ thích một phim lịch sử D (giống A, B, C). Sở thích này hoàn toàn dựa trên lịch sử của người dùng. User-based thì lại dự đoán dựa vào rating của “hàng xóm” (những người giống bạn), có thể họ có “gu” chồng lấn nhưng chưa chắc hoàn toàn giống.

1.2.2 Model-Based Collaborative Filtering

Xây dựng mô hình tổng quát trước từ dữ liệu (giống supervised hoặc unsupervised learning). Có hai pha rõ ràng: training/model-building (huấn luyện), và prediction (dự đoán). Trong collaborative filtering, bài toán thực chất là một dạng matrix completion: Ma trận $m \times n$, mỗi hàng là user, mỗi cột là item, mỗi ô là rating (rất nhiều ô bị thiếu).

Mục tiêu: Dùng một mô hình học được để dự đoán (fill) các giá trị còn thiếu. So sánh với bài toán classification/regression truyền thống:

- Vai trò feature/class không còn rõ ràng
 - Classification: Có phân biệt rõ cột “feature” (đầu vào) và cột “class/label” (đầu ra, cột phụ thuộc).

- Collaborative filtering: Mỗi ô trong ma trận rating đều có thể là giá trị cần dự đoán, không phân biệt rõ “feature” và “label” — bất cứ entry nào cũng có thể bị thiếu (missing).
- Khái niệm train/test data không còn rạch ròi
 - Classification: Dữ liệu train/test phân theo từng dòng (row).
 - Collaborative filtering: Giá trị missing (cần dự đoán) phân bố rải rác khắp ma trận, nên “train” là các ô đã biết, “test” là các ô chưa biết — không còn theo dòng/cột cố định.
- Cách hoán đổi user/item
 - Trong collaborative filtering, có thể áp dụng thuật toán trên cả ma trận gốc (user-item) hoặc ma trận chuyển vị (item-user), dẫn tới các phiên bản “user-based” hoặc “item-based” của cùng một phương pháp (ví dụ: user-based kNN, item-based kNN).

1.3 Content-based Recommender Systems

1.3.1 Đặc điểm chính

Dựa vào thuộc tính mô tả (content/metadata) của từng sản phẩm (item) để gợi ý sản phẩm mới cho user. Nói cách khác, hệ thống này tìm hiểu bạn thích gì từ các hành vi quá khứ của bạn, rồi gợi ý những thứ có đặc điểm tương tự. Các thuộc tính này có thể là: thể loại phim, diễn viên, đạo diễn, từ khóa, chủ đề, tác giả, mô tả sản phẩm. . .

1.3.2 Các thành phần cơ bản của Content-Based Systems

Tiền xử lý (Preprocessing) và Trích xuất đặc trưng (Feature Extraction)
Feature extraction là bước chuyển dữ liệu thô (chủ yếu là text, meta-data) thành dạng vector đặc trưng dùng cho máy học.

- Với văn bản: Trích xuất từ khóa, thể loại, tác giả, chủ đề, hoặc dùng kỹ thuật như TF-IDF, word embedding. . .
- Với nhạc: Genre, tempo, nghệ sĩ, mô tả kỹ thuật (ví dụ Pandora).
- Với sản phẩm: Giá, màu sắc, thương hiệu, mô tả. . .

Hiệu quả của content-based recommender phụ thuộc rất lớn vào việc bạn có trích xuất được đặc trưng “phân biệt tốt” hay không. Feature representation and cleaning: Chuyển dữ liệu mô tả thô (text, review, mô tả sản phẩm, trang web...) thành vector số học sạch, cô đọng, loại bỏ nhiễu, giúp mô hình học máy xử lý hiệu quả và chính xác hơn.

- Stop-word Removal (loại bỏ từ dừng): Loại bỏ những từ quá phổ biến, không mang ý nghĩa phân biệt sản phẩm, như “a”, “an”, “the”, “is”, “are”, “of”, “in”, “on”, v.v. Những từ này xuất hiện ở mọi tài liệu, không giúp phân biệt sản phẩm nào với sản phẩm nào.
- Stemming (Gốc từ): Đưa các biến thể của từ về một gốc chung, giúp giảm số lượng feature và tăng khả năng nhận diện chủ đề. Ví dụ: “hoped”, “hoping”, “hopes” thành “hope”. Lưu ý: Đôi khi có thể gây lỗi nghĩa (ví dụ “hop” là “nhảy lò cò” không liên quan đến “hope” là “hy vọng”).
- Phrase Extraction (Trích cụm từ): Một số cụm từ (phrase) có nghĩa đặc biệt, không nên tách rời (“hot dog”)

Sau khi hoàn thành xong các bước này, mỗi tài liệu (sản phẩm) sẽ được biểu diễn thành một vector gồm các từ khóa (term vector). Mỗi thành phần trong vector là một từ hoặc cụm từ, giá trị thường là số lần xuất hiện hoặc trọng số (weighted frequency) Chuẩn hóa trọng số từ khóa: TF-IDF Những từ phổ biến trong toàn bộ kho sản phẩm (như “story”, “book”, “movie”) sẽ xuất hiện ở mọi nơi, không giúp phân biệt sản phẩm. Nếu giữ nguyên tần suất, các từ này sẽ có trọng số rất cao → gây nhiễu cho mô hình. Inverse Document Frequency (IDF): Giảm trọng số cho các từ xuất hiện ở nhiều tài liệu. Từ càng hiếm (n_i nhỏ), idf càng lớn thì trọng số càng cao (tốt để phân biệt).

$$id_i = \log(n/n_i)$$

Nếu một từ xuất hiện cực nhiều trong một tài liệu (do spam hoặc văn bản dài), cũng không nên để nó áp đảo hoàn toàn. Dùng hàm giảm ảnh hưởng:

$$f(x_i) = \sqrt{x_i}$$
$$f(x_i) = \log(x_i)$$

Có thể bỏ qua bước này (đặt $f(x_i)=x_i$) Kết hợp lại: TF-IDF

$$h(x_i) = f(x_i)id_i$$

1.4 Knowledge-based recommendation system

Knowledge-based recommendation system là hệ thống gợi ý dựa trên kiến thức về sản phẩm/dịch vụ và nhu cầu, ưu tiên của người dùng. Khác với collaborative filtering hay content-based filtering, hệ thống này không dựa vào lịch sử đánh giá hoặc hành vi người dùng, mà dựa vào các quy tắc (rules), mối quan hệ logic và thông tin chuyên môn đã được định nghĩa sẵn.

Cả hệ thống dựa trên nội dung (content-based) và hệ thống cộng tác (collaborative) đều đòi hỏi một lượng dữ liệu đáng kể về các trải nghiệm mua hàng và đánh giá trong quá khứ. Ví dụ, các hệ thống cộng tác cần có một ma trận đánh giá được điền đầy đủ ở mức độ nhất định để có thể đưa ra các khuyến nghị trong tương lai. Trong những trường hợp dữ liệu sẵn có còn hạn chế, các khuyến nghị đưa ra thường kém chất lượng, hoặc không bao phủ được toàn bộ các tổ hợp người dùng - mặt hàng. Vấn đề này còn được gọi là vấn đề cold-start (khởi đầu lạnh). Các hệ thống khác nhau sẽ bị ảnh hưởng ở các mức độ khác nhau bởi vấn đề này. Chẳng hạn, các hệ thống cộng tác bị ảnh hưởng nhiều nhất và không thể xử lý tốt các mặt hàng mới hoặc người dùng mới. Các hệ thống gợi ý dựa trên nội dung có khả năng xử lý các mặt hàng mới tốt hơn phần nào, nhưng vẫn không thể đưa ra khuyến nghị cho người dùng mới.

2 GCN

2.1 Giới thiệu

GCN (Graph Convolutional Network) là một loại mạng nơ-ron sâu (deep neural network) được thiết kế để xử lý dữ liệu có cấu trúc đồ thị, tức là dữ liệu mà các đối tượng (node) có mối liên kết với nhau (edge). GCN giống như một phiên bản “tích chập” (convolution) dành cho đồ thị, thay vì dành cho ảnh (như CNN).

2.2 Cách hoạt động

Mỗi node (nút) trong đồ thị sẽ cập nhật biểu diễn (vector đặc trưng) của mình bằng cách tổng hợp (aggregate) thông tin từ các node hàng xóm (neighbor, các node kết nối trực tiếp).

Việc tổng hợp này lặp lại qua nhiều lớp (layer), giúp GCN có khả năng “nhìn xa” – không chỉ học được thông tin của các node lân cận trực tiếp

mà còn của các node xa hơn (neighbors của neighbors). GCN rất mạnh cho các bài toán học bán giám sát (semi-supervised learning) trên đồ thị, ví dụ như phân loại node (node classification) khi chỉ một phần các node được gán nhãn.

Ví dụ:

Giả sử đồ thị như sau:

- Node A (có nhãn: fan bóng đá)
- Node B (không nhãn), kết nối với A
- Node C (không nhãn), kết nối với B

Lan truyền qua các lớp:

- Lớp 1: B nhận thông tin embedding từ A (fan bóng đá) $\rightarrow$ embedding của B bắt đầu giống với A. C chỉ nhận từ B (chưa rõ), chưa ảnh hưởng nhiều.
- Lớp 2: B đã giống A, giờ C nhận embedding từ B $\rightarrow$ C bắt đầu có embedding giống fan bóng đá dù không kết nối trực tiếp với node có nhãn.

Nếu C lại kết nối với nhiều node khác đều liên quan “fan bóng đá”, embedding của nó sẽ càng rõ xu hướng này.

GCN cho phép embedding của node chưa gán nhãn dần bị “kéo” về phía embedding của nhóm node có nhãn mà chúng kết nối đến, nhờ đó lan truyền “thông tin nhãn” gián tiếp ra toàn mạng, giúp dự đoán nhãn tốt kể cả khi số node có nhãn rất ít.

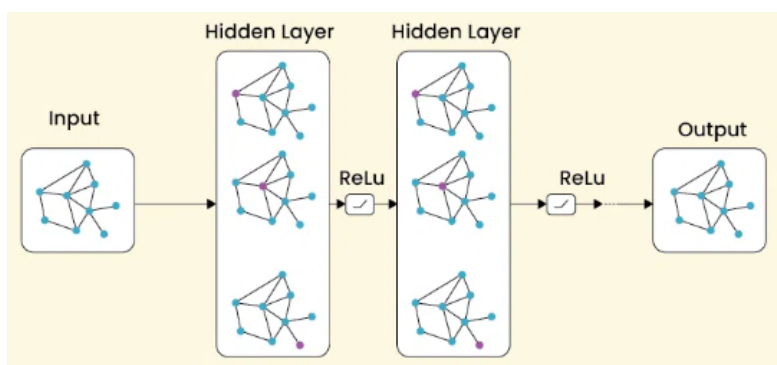


Figure 1: Graph Convolutional Networks

Giải thích hình minh họa

- Đi từng bước từ trái qua phải: Input (Đầu vào): Một đồ thị gồm nhiều node (vòng tròn nhỏ) và các cạnh (đường nối). Mỗi node có đặc trưng ban đầu (input vector).
- Hidden layer 1 (Lớp ẩn 1): Mỗi node nhận thông tin từ các node lân cận để cập nhật vector đặc trưng mới cho mình (màu sắc, vị trí node có thể thay đổi, thể hiện vector đã thay đổi). Đây là một bước “lan truyền thông tin” đầu tiên.
- ReLU (Hàm kích hoạt): Sau khi tổng hợp thông tin từ hàng xóm, vector đặc trưng mới sẽ đi qua hàm kích hoạt phi tuyến (ReLU), giúp mô hình học các quan hệ phức tạp hơn.
- Hidden layer 2 (Lớp ẩn 2): Lặp lại: Mỗi node lại tiếp tục tổng hợp thông tin từ neighbors (lúc này embedding của neighbor đã được update từ layer trước), giúp node học được thông tin từ hàng xóm xa hơn.
- ReLU: Lại qua hàm kích hoạt ReLU.
- Output (Đầu ra): Sau nhiều lớp, mỗi node đã “tích lũy” thông tin tổng hợp từ toàn bộ vùng lân cận trong đồ thị.
- Embedding cuối cùng của từng node có thể dùng cho các tác vụ như dự đoán nhãn, phân loại node,...

2.3 Ưu điểm

- Khai thác tối đa cấu trúc đồ thị
 - Tận dụng thông tin liên kết giữa các node (neighbor) thay vì chỉ đặc trưng của node.
 - Tốt cho dữ liệu dạng network: mạng xã hội, mạng protein, recommendation,...
- Lan truyền thông tin hiệu quả: Node học được đặc trưng không chỉ từ bản thân mà còn từ cả hàng xóm nhiều cấp, đặc biệt mạnh trong học bán giám sát (semi-supervised learning) với dữ liệu gán nhãn ít.
- Tự động học đặc trưng: Không cần trích xuất đặc trưng thủ công, GCN tự học embedding phù hợp với bài toán.
- Tổng quát hóa tốt: Có thể áp dụng cho node mới, graph mới nhờ cơ chế chia sẻ tham số.
- Hiệu quả thực nghiệm vượt trội: Được chứng minh tốt hơn các phương pháp truyền thống trên nhiều benchmark.

2.4 Nhược điểm

- Khó mở rộng với đồ thị cực lớn (Scalability): Khi số node và cạnh rất lớn, quá trình lan truyền neighbor sẽ tốn bộ nhớ và thời gian (bottleneck về tính toán và RAM).
- Vấn đề over-smoothing (Làm phẳng embedding): Khi tăng số lớp, embedding của các node dễ bị “hòa trộn” quá mức, mất đi sự khác biệt → các node khác class có embedding gần giống nhau.
- Phụ thuộc nhiều vào cấu trúc đồ thị: Nếu đồ thị “nhiều” (kết nối không phản ánh ngữ nghĩa), GCN có thể học sai lệch.
- Chưa tận dụng tốt đặc trưng cạnh (edge features): GCN truyền thống chủ yếu dùng thông tin node và adjacency, ít dùng thông tin đặc trưng phức tạp trên cạnh (weight, type).
- Hạn chế khi đồ thị dị hướng (directed graph) hoặc dị thể (heterogeneous graph): GCN cơ bản phù hợp nhất cho đồ thị vô hướng, đồng nhất;

muốn xử lý phức tạp hơn phải mở rộng kiến trúc (biến thể như GAT, RGCN,...).

3 LightGCN

3.1 Giới thiệu

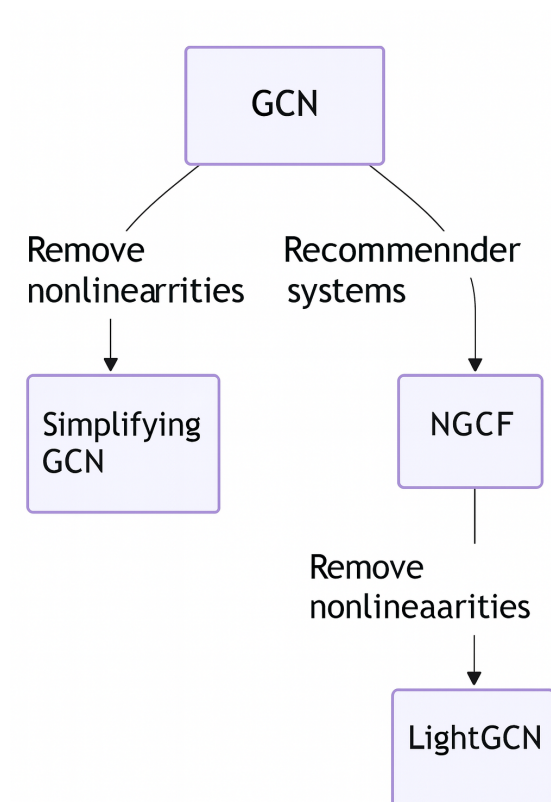


Figure 2: Sơ đồ phát triển các mô hình GCN cho hệ thống gợi ý

Mạng Nơ-ron Tích chập Đồ thị (Graph Convolutional Network - GCN) đã trở thành phương pháp mới đạt hiệu quả cao nhất cho bài toán lọc cộng tác (collaborative filtering). Tuy nhiên, lý do tại sao GCN lại hiệu quả cho hệ thống gợi ý vẫn chưa được hiểu rõ. Các nghiên cứu hiện tại khi áp dụng GCN cho hệ thống gợi ý thường thiếu các phân tích ablation kỹ lưỡng về GCN – vốn ban đầu được thiết kế cho các bài toán phân loại đồ thị và trang bị nhiều phép toán của mạng nơ-ron. Tuy nhiên, qua thực nghiệm, chúng

tôi nhận thấy rằng hai thiết kế phổ biến nhất trong GCN – chuyển đổi đặc trưng (feature transformation) và kích hoạt phi tuyến (nonlinear activation) – thực tế lại đóng vai trò rất ít đối với hiệu suất của collaborative filtering. Thậm chí, việc đưa các thành phần này vào còn làm quá trình huấn luyện trở nên khó khăn hơn và làm giảm hiệu quả của hệ thống gợi ý.

Do vậy, một mô hình mới có tên là LightGCN, chỉ giữ lại thành phần cốt lõi nhất của GCN — đó là bước tổng hợp thông tin từ hàng xóm (neighborhood aggregation) — dành cho bài toán collaborative filtering. Cụ thể, LightGCN học embedding của user và item bằng cách lan truyền tuyến tính các embedding này trên đồ thị tương tác user-item, sau đó sử dụng tổng có trọng số của các embedding thu được ở tất cả các lớp làm embedding cuối cùng. Mô hình này rất đơn giản, tuyến tính và gọn gàng, giúp việc triển khai và huấn luyện trở nên dễ dàng hơn nhiều, đồng thời đạt được mức cải thiện đáng kể (trung bình khoảng 16,0% so với mô hình Neural Graph Collaborative Filtering - NGCF, vốn là một mô hình recommender dựa trên GCN tiên tiến) trong cùng một điều kiện thực nghiệm. Ngoài ra, chúng tôi còn cung cấp các phân tích sâu hơn về tính hợp lý của LightGCN từ cả góc độ lý thuyết và thực nghiệm.

Để giảm bớt tình trạng quá tải thông tin trên web, hệ thống gợi ý đã được triển khai rộng rãi nhằm thực hiện việc lọc thông tin cá nhân hóa. Cốt lõi của hệ thống gợi ý là dự đoán liệu một người dùng sẽ tương tác với một sản phẩm hay không, ví dụ như nhấp chuột, đánh giá, mua hàng hoặc các hình thức tương tác khác. Do đó, collaborative filtering (CF – lọc cộng tác), vốn tập trung khai thác các tương tác người dùng – sản phẩm trong quá khứ để dự đoán hành vi tương lai, vẫn luôn là nhiệm vụ nền tảng hướng tới xây dựng hệ thống gợi ý cá nhân hóa hiệu quả.

Cách tiếp cận phổ biến nhất cho CF là học các đặc trưng tiềm ẩn (hay còn gọi là embedding) để biểu diễn cho mỗi người dùng và mỗi sản phẩm, sau đó thực hiện dự đoán dựa trên các vector embedding này. Matrix factorization là một trong những mô hình sớm nhất thuộc loại này, khi trực tiếp ánh xạ một ID duy nhất của người dùng thành embedding của họ. Sau này, một số nghiên cứu phát hiện rằng việc bổ sung lịch sử tương tác của người dùng (bên cạnh ID) vào input sẽ giúp cải thiện chất lượng embedding. Ví dụ, SVD++ chứng minh lợi ích của việc sử dụng lịch sử tương tác người dùng trong dự đoán điểm số đánh giá, còn mô hình Neural Attentive Item Similarity (NAIS) phân biệt tầm quan trọng của từng sản phẩm trong lịch sử tương tác và cho thấy hiệu quả trong bài toán xếp hạng sản phẩm.

Ví dụ về Matrix Factorization

Giả sử bạn có một hệ thống đề xuất phim. Mỗi người dùng sẽ có một embedding đặc trưng cho sở thích của họ, và mỗi bộ phim cũng có một embedding đại diện cho nội dung của nó. Khi một người dùng chưa xem một bộ phim, hệ thống có thể dự đoán mức độ yêu thích của họ bằng cách tính toán sản phẩm của hai embedding này.

Ví dụ về SVD++

Giả sử một người dùng đã xem nhiều phim hành động và đánh giá cao chúng. SVD++ tận dụng thông tin này bằng cách xem xét không chỉ embedding của người dùng mà còn các bộ phim họ đã tương tác trước đó, giúp dự đoán chính xác hơn khi gặp một bộ phim hành động mới.

Ví dụ về NAIS

Không phải tất cả các bộ phim mà một người dùng từng xem đều có ảnh hưởng như nhau đến sở thích của họ. NAIS sử dụng cơ chế attention để xác định mức độ quan trọng của từng bộ phim trước đó trong việc dự đoán liệu họ có thích một bộ phim mới hay không.

Khi nhìn nhận dưới góc độ đồ thị tương tác người dùng – sản phẩm, những cải tiến này có thể xem như việc tận dụng cấu trúc đồ thị con (subgraph) xung quanh một người dùng — cụ thể là các hàng xóm một bước (one-hop neighbors) — để cải thiện quá trình học embedding.

Để khai thác sâu hơn cấu trúc đồ thị con với các hàng xóm nhiều bước (high-hop neighbors), Wang và cộng sự gần đây đã đề xuất NGCF và đạt được hiệu suất hàng đầu cho CF. NGCF lấy cảm hứng từ Graph Convolution Network (GCN), áp dụng quy tắc lan truyền tương tự để tinh chỉnh embedding: chuyển đổi đặc trưng (feature transformation), tổng hợp thông tin hàng xóm (neighborhood aggregation) và kích hoạt phi tuyến (nonlinear activation). Mặc dù NGCF đã cho thấy kết quả đầy hứa hẹn, chúng tôi cho rằng thiết kế của mô hình này khá nặng nề và cồng kềnh — nhiều phép toán được kế thừa trực tiếp từ GCN mà không có sự đánh giá kỹ lưỡng về tính cần thiết. Do đó, không phải thành phần nào cũng thực sự hữu ích cho bài toán CF. Cụ thể, GCN ban đầu được đề xuất để phân loại node trên đồ thị có thuộc tính, nơi mỗi node có bộ đặc trưng (feature) phong phú làm đầu vào; trong khi đó, ở đồ thị tương tác user-item cho CF, mỗi node (user hoặc item) chỉ được mô tả bởi một ID one-hot, vốn không mang ý nghĩa nào ngoài vai trò định danh. Trong trường hợp này, khi embedding dựa trên ID được dùng làm input, việc thực hiện nhiều tầng chuyển đổi đặc trưng phi tuyến — vốn là chìa khóa thành công của các mạng neural hiện đại [16] — lại không mang lại lợi ích, mà thậm chí còn làm tăng độ khó khi huấn luyện mô hình.

Để kiểm chứng nhận định này, những nhà nghiên cứu đã thực hiện các nghiên cứu ablation mở rộng trên NGCF. Qua các thí nghiệm được kiểm soát nghiêm ngặt (cùng chia tập dữ liệu và cùng giao thức đánh giá), rút ra kết luận rằng hai phép toán được kế thừa từ GCN — chuyển đổi đặc trưng và kích hoạt phi tuyến — không đóng góp gì vào hiệu quả của NGCF. Thậm chí, việc loại bỏ chúng còn dẫn đến cải thiện độ chính xác đáng kể. Điều này phản ánh vấn đề của việc thêm các phép toán không phù hợp vào mạng neural đồ thị, không chỉ không mang lại lợi ích mà còn làm giảm hiệu quả của mô hình.

Từ những phát hiện thực nghiệm này, mô hình mới có tên LightGCN, chỉ giữ lại thành phần cốt lõi nhất của GCN — tổng hợp thông tin hàng xóm (neighborhood aggregation) — cho collaborative filtering. Cụ thể, sau khi gán cho mỗi user (hoặc item) một embedding theo ID, chúng ta lan truyền các embedding này trên đồ thị tương tác user-item để tinh chỉnh chúng. Sau đó, chúng ta kết hợp các embedding học được ở các tầng lan truyền khác nhau bằng tổng có trọng số để tạo thành embedding cuối cùng dùng cho dự đoán.

3.2 Kiến trúc mô hình

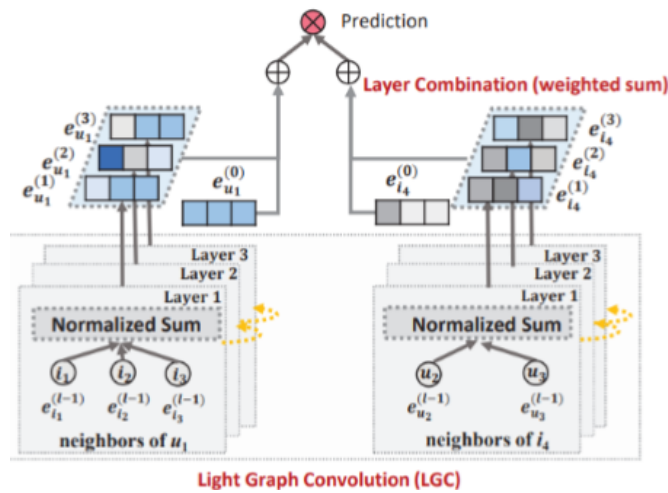


Figure 3: Kiến trúc LightGCN

Mô tả:

1. Lan truyền embedding qua các layer (Light Graph Convolution – LGC)

- (a) Khởi tạo embedding cho user và item: Mỗi user (ví dụ u_1) và mỗi item (ví dụ i_4) ban đầu được gán một vector embedding ($e_{u_1}^{(0)}$)—các ô vuông đầu tiên trong hình.
- (b) Lan truyền embedding qua nhiều lớp (layer):
 - i. Ở mỗi layer, embedding của một node (user hoặc item) được cập nhật dựa trên các hàng xóm của nó (neighbors).
 - ii. Cập nhật embedding: Ở mỗi lớp, embedding mới là tổng chuẩn hóa (normalized sum) các embedding của hàng xóm từ layer trước (xem khung "Normalized Sum").
2. Lặp lại quá trình trên qua nhiều layer: Mỗi layer, thông tin lan truyền càng xa hơn trên đồ thị user-item, giúp embedding nắm bắt được các tương tác gián tiếp (ví dụ: bạn của bạn cũng thích món đồ này).
3. Kết hợp embedding từ nhiều layer: Thay vì chỉ dùng embedding ở layer cuối, LightGCN kết hợp các embedding ở tất cả các layer (Layer Combination). Embedding cuối cùng của user/item là tổng trọng số (hoặc trung bình) embedding ở các layer 0, 1, 2, 3 (như hình minh họa bằng nhiều hàng ô vuông).
4. Dự đoán tương tác (Prediction):
 - (a) Để dự đoán user u_i có thích item i_4 không, lấy embedding cuối cùng của u_i và i_4 , tính dot product (tích vô hướng).
 - (b) Nếu điểm số cao thì xác suất tương tác cao, gợi ý cho user.

3.3 Light Graph Convolution (LGC) trong LightGCN

3.3.1 Công thức lan truyền (propagation rule)

$$\begin{aligned}e_u^{(k+1)} &= \sum_{i \in N_u} \frac{1}{\sqrt{|N_i|}\sqrt{|N_u|}} e_i^{(k)}, \\e_i^{(k+1)} &= \sum_{u \in N_i} \frac{1}{\sqrt{|N_i|}\sqrt{|N_u|}} e_u^{(k)}.\end{aligned}$$

Trong đó:

- N_u : Tập các item đã được user u tương tác (hàng xóm của user).
- N_i : Tập các user đã tương tác với item i .

- $|N_u|$: Số lượng các item đã được user u tương tác.
- $|N_i|$: Số lượng các user đã tương tác với item i .

Ta thấy rằng không lấy self-connection (không tính chính node đó vào hàng xóm), chỉ cộng embedding từ các node kết nối trực tiếp (đây là điểm khác biệt lớn với GCN gốc). Không dùng nonlinear activation $\rightarrow$ đơn giản hơn, giảm chi phí tính toán.

3.3.2 Layer Combination (Kết hợp nhiều lớp embedding)

$$e_u = \sum_{k=0}^K \alpha_k e_u^{(k)}; \quad e_i = \sum_{k=0}^K \alpha_k e_i^{(k)},$$

Trong đó:

- e_u : Embedding cuối cùng của user u
- e_i : Embedding cuối cùng của item i
- α_k : Trọng số (weight) của embedding ở tầng k (có thể học được hoặc lấy đều nhau)
- $e_u^{(k)}$: Embedding của user u sau layer k
- $e_i^{(k)}$: Embedding của item i sau layer k

Chỉ embedding ở layer 0 (ban đầu) là tham số huấn luyện. Các layer sau đều được lan truyền theo công thức trên, không cần thêm tham số mới. Sau khi lan truyền qua K layer, embedding cuối cùng là tổng trọng số (hoặc trung bình) của embedding từ tất cả các layer.

Lý do kết hợp nhiều layer:

- Tránh over-smoothing: Nếu chỉ lấy embedding ở layer cuối, embedding của tất cả các node sẽ giống nhau dần (mất cá thể). Kết hợp các layer giúp giữ được đa dạng thông tin.
- Mỗi layer học ra các mức ý nghĩa khác nhau:
 - Layer đầu: thông tin trực tiếp (user từng tương tác item nào)
 - Layer sau: thông tin gián tiếp (user nào cùng từng tương tác item giống user này)
- Kết hợp embedding các layer sẽ có tác dụng tương tự như self-connection trong GCN gốc, nhưng không làm mô hình phức tạp thêm.

3.3.3 Dự đoán của mô hình LightGCN (Model Prediction)

$$\hat{y}_{ui} = e_u^T e_i$$

Nếu y_{ui} cao thì hệ thống dự đoán user u sẽ quan tâm hoặc tương tác với item i (ví dụ: click, mua, like). Dùng điểm này để xếp hạng các item và gợi ý item phù hợp cho từng user.

3.4 Dạng ma trận LightGCN

3.4.1 Ma trận tương tác user-item ($\mathbf{R}$)

$$\mathbf{R} \in \mathbb{R}^{M \times N}$$

Trong đó:

- M : Số lượng user
- N : Số lượng item
- $R_{ui} = 1$ nếu user u đã tương tác với item i , ngược lại là 0

3.4.2 Ma trận kề của đồ thị user-item ($\mathbf{A}$)

$\mathbf{A}$ là ma trận vuông $(M + N) \times (M + N)$, kết hợp user và item vào cùng một không gian:

$$\begin{bmatrix} 0 & R \\ R^T & 0 \end{bmatrix}$$

Trong đó:

- Các block 0 (trên-trái, dưới-phải): Không có liên kết giữa user-user hoặc item-item
- Block R : Liên kết user $\rightarrow$ item
- Block R^T : Liên kết item $\rightarrow$ user

3.4.3 Embedding khởi tạo ($E^{(0)}$)

$$E^{(0)} \in \mathbb{R}^{(M+N) \times T}$$

Trong đó:

- T: Số chiều embedding
- Dòng $1 \rightarrow M$ là embedding user, dòng $M+1 \rightarrow M+N$ là embedding item

3.4.4 Công thức lan truyền embedding qua các layer

$$E^{k+1} = D^{\frac{-1}{2}} \times A \times D^{\frac{-1}{2}} \times E^{(k)}$$

Trong đó:

- D là ma trận bậc (degree matrix): D_{ii} bằng số liên kết (degree) của node i trên đồ thị A.

3.4.5 Kết hợp nhiều layer để ra embedding cuối cùng

$$\begin{aligned} E &= \alpha_0 \times E^{(0)} + \alpha_1 \times E^{(1)} + \dots + \alpha_k \times E^{(k)} \\ &= \alpha_0 \times E^{(k)} + \alpha_0 \times \tilde{A} \times E^{(0)} + \dots + \alpha_k \times \tilde{A} \times E^{(0)} \end{aligned}$$

Trong đó:

- α_k : Trọng số cho mỗi layer (thường lấy đều nhau: $\alpha_k = \frac{1}{K+1}$)
- Mỗi $E^k = \tilde{A} \times E^{(0)}$ tức là embedding sau k lần lan truyền qua đồ thị.

3.5 Huấn luyện mô hình

Mô hình LightGCN có cấu trúc đơn giản, các tham số cần huấn luyện chỉ gồm các vector embedding ban đầu ($E^{(0)}$) của user và item, giống hệt như trong mô hình Matrix Factorization cổ điển. Quá trình huấn luyện sử dụng hàm mất mát Bayesian Personalized Ranking (BPR loss), một hàm pairwise loss phổ biến trong các hệ thống gợi ý dựa trên implicit feedback.

$$L_{BPR} = - \sum_{u=1}^M \sum_{i \in \mathcal{N}_u} \sum_{j \in \mathcal{N}_u} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|E^{(0)}\|^2$$

BPR loss khuyến khích điểm dự đoán của các item mà người dùng đã tương tác cao hơn so với các item chưa từng tương tác, đồng thời áp dụng thêm L2 regularization để giảm nguy cơ overfitting. Mô hình sử dụng tối ưu hóa Adam kết hợp với mini-batch để đảm bảo khả năng mở rộng khi áp dụng trên tập dữ liệu lớn.

Khác với nhiều mô hình GCN truyền thống hoặc NGCF, LightGCN không sử dụng dropout bởi vì bản thân nó không có các ma trận biến đổi đặc trưng (feature transformation weight matrix); việc áp dụng L2 regularization lên các embedding là đủ để kiểm soát overfitting. Ngoài ra, LightGCN còn tối giản quy trình huấn luyện khi không cần điều chỉnh các siêu tham số phức tạp như dropout ratio hoặc chuẩn hóa embedding từng tầng, giúp mô hình dễ huấn luyện, dễ mở rộng và tinh chỉnh hơn các mô hình trước đây.

Trong việc kết hợp các embedding từ nhiều tầng, LightGCN mặc định sử dụng trung bình cộng và không cần học thêm các hệ số trọng số (α), bởi thử nghiệm cho thấy việc này không cải thiện hiệu quả mô hình một cách đáng kể. Tuy vẫn còn nhiều hướng cải tiến như hard negative sampling, adversarial sampling hoặc cá nhân hóa hệ số α , LightGCN vẫn nổi bật nhờ thiết kế tối giản nhưng hiệu quả, mang lại hiệu suất tốt mà không cần cấu hình quá nhiều siêu tham số phức tạp.

3.6 So sánh hiệu năng: LightGCN và NGCF

Dataset		Gowalla		Yelp2018		Amazon-Book	
Layer #	Method	recall	ndcg	recall	ndcg	recall	ndcg
1 Layer	NGCF	0.1556	0.1315	0.0543	0.0442	0.0313	0.0241
	LightGCN	0.1755(+12.79%)	0.1492(+13.46%)	0.0631(+16.20%)	0.0515(+16.51%)	0.0384(+22.68%)	0.0298(+23.65%)
2 Layers	NGCF	0.1547	0.1307	0.0566	0.0465	0.0330	0.0254
	LightGCN	0.1777(+14.84%)	0.1524(+16.60%)	0.0622(+9.89%)	0.0504(+8.38%)	0.0411(+24.54%)	0.0315(+24.02%)
3 Layers	NGCF	0.1569	0.1327	0.0579	0.0477	0.0337	0.0261
	LightGCN	0.1823(+16.19%)	0.1555(+17.18%)	0.0639(+10.38%)	0.0525(+10.06%)	0.0410(+21.66%)	0.0318(+21.84%)
4 Layers	NGCF	0.1570	0.1327	0.0566	0.0461	0.0344	0.0263
	LightGCN	0.1830(+16.56%)	0.1550(+16.80%)	0.0649(+14.58%)	0.0530(+15.02%)	0.0406(+17.92%)	0.0313(+18.92%)

Figure 4: So sánh hiệu năng giữa LightGCN và NGCF ở những layer khác nhau

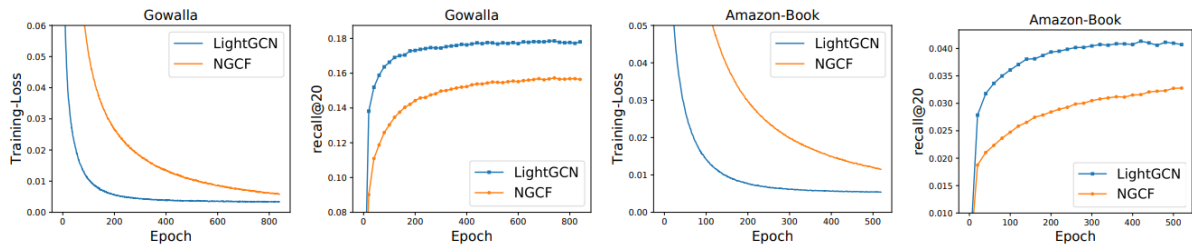


Figure 5: Các đường cong huấn luyện của LightGCN và NGCF, được đánh giá bằng độ mất mát trong quá trình huấn luyện và độ nhớ kiểm tra sau mỗi 20 epoch

Ta có thể rút ra một số nhận xét như sau:

- LightGCN luôn cho recall và ndcg cao hơn NGCF, chênh lệch khá lớn.
- LightGCN cũng vượt luôn cả biến thể NGCF-fn (loại bỏ feature transformation và nonlinear activation), chứng tỏ sự đơn giản hóa của LightGCN là hợp lý.
- Tăng số layer giúp cải thiện hiệu năng, nhưng hiệu quả gia tăng dần nhỏ lại (diminishing return). Layer 1 tăng mạnh nhất, layer 3 là đủ tốt trong đa số trường hợp.
- Đồ thị training loss và recall: LightGCN luôn đạt training loss thấp hơn (fit dữ liệu train tốt hơn), đồng thời recall test cũng cao hơn, chứng tỏ LightGCN vừa học tốt vừa tổng quát hóa tốt.
- NGCF dù có thể giảm training loss (bằng cách tăng learning rate), nhưng recall test không cải thiện do dễ overfit hoặc gặp các giải pháp "trivial" (không thực sự học tốt).
- LightGCN huấn luyện dễ hơn, ít rủi ro overfitting, không cần nhiều thành phần phức tạp như NGCF (không self-connection, không feature interaction, không nonlinear, không dropout).

3.7 So sánh hiệu năng với các mô hình tiên tiến khác

Khi so sánh với các mô hình như NGCF, Mult-VAE, GRMF, GRMF-norm, ta nhận thấy được:

Dataset	Gowalla		Yelp2018		Amazon-Book	
Method	recall	ndcg	recall	ndcg	recall	ndcg
NGCF	0.1570	0.1327	0.0579	0.0477	0.0344	0.0263
Mult-VAE	0.1641	0.1335	0.0584	0.0450	0.0407	0.0315
GRMF	0.1477	0.1205	0.0571	0.0462	0.0354	0.0270
GRMF-norm	0.1557	0.1261	0.0561	0.0454	0.0352	0.0269
LightGCN	0.1830	0.1554	0.0649	0.0530	0.0411	0.0315

Figure 6: So sánh hiệu năng với các mô hình khác

- LightGCN luôn đạt điểm cao nhất trên mọi bộ dữ liệu (cả recall lẫn ndcg).
- Mult-VAE là baseline mạnh nhất sau LightGCN (vẫn thua khá xa trên Gowalla).
- GRMF/GRMF-norm mạnh hơn MF, nhưng yếu hơn các mô hình graph/deep khác.
- GRMF-norm chỉ tốt hơn GRMF trên Gowalla, các dataset khác không có lợi thế rõ.

Tóm lại, LightGCN chứng minh rằng thiết kế đơn giản, tối ưu hoá lan truyền thông tin hàng xóm mà không cần feature transformation, nonlinear activation, hay self-connection vẫn có thể vượt qua mọi baseline khác kể cả các mô hình phức tạp (NGCF, VAE).

3.8 Tầm quan trọng của các thành phần trong LightGCN

3.8.1 Layer Combination

Ta tiến hành so sánh LightGCN và LightGCN-single:

- LightGCN: Kết hợp embedding của mọi layer ($E = \sum \alpha_k E^{(k)}$)
- LightGCN-single: Chỉ dùng embedding của layer cuối (E^k), không kết hợp nhiều tầng.

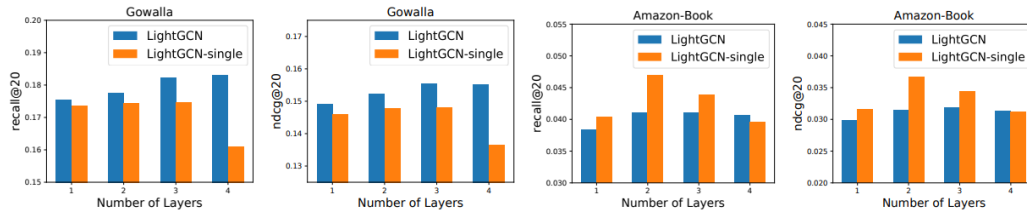


Figure 7: So sánh hiệu suất của LightGCN và LightGCN-single trên sự thay đổi layer

Ta có thể rút ra một số nhận xét như sau:

- LightGCN-single:
 - Khi tăng số layer từ 1 → 2, hiệu năng tăng (vì thêm thông tin hàng xóm bậc cao).
 - Khi tăng layer lên 3 trở đi thì hiệu năng của mô hình giảm do over-smoothing.
- LightGCN (có layer combination):
 - Hiệu năng tăng dần đều khi tăng số layer (thậm chí layer 4 vẫn không giảm). Là do kết hợp nhiều tầng giúp tận dụng thông tin bậc cao mà không bị “over-smoothing” như chỉ lấy mỗi layer cuối.
 - LightGCN vượt trội LightGCN-single trên Gowalla (lớn, nhiều tương tác). Trên Amazon-Book, LightGCN-single (layer 2) tốt nhất.

3.8.2 Tác động của Symmetric Sqrt Normalization trong LightGCN

Khi tổng hợp embedding từ hàng xóm, LightGCN dùng hệ số. Đây gọi là chuẩn hóa căn bậc hai đối xứng:

$$\frac{1}{\sqrt{|N_u|} \times \sqrt{|N_i|}}$$

Trong đó:

- $|N_u|$: Số hàng xóm của node nhận (user)

- $|N_i|$: Số hàng xóm của node gửi (item)

Ta tiến hành so sánh các kiểu chuẩn hóa (Normalization) trong LightGCN

Dataset	Gowalla		Yelp2018		Amazon-Book	
Method	recall	ndcg	recall	ndcg	recall	ndcg
LightGCN- L_1 -L	0.1724	0.1414	0.0630	0.0511	0.0419	0.0320
LightGCN- L_1 -R	0.1578	0.1348	0.0587	0.0477	0.0334	0.0259
LightGCN- L_1	0.159	0.1319	0.0573	0.0465	0.0361	0.0275
LightGCN-L	0.1589	0.1317	0.0619	0.0509	0.0383	0.0299
LightGCN-R	0.1420	0.1156	0.0521	0.0401	0.0252	0.0196
LightGCN	0.1830	0.1554	0.0649	0.0530	0.0411	0.0315

Figure 8: So sánh các kiểu chuẩn hóa (Normalization) trong LightGCN

Chú thích:

- LightGCN: Chuẩn hóa đối xứng “chuẩn”
- LightGCN- L_1 -L: Chỉ chuẩn hóa trái với L_1 -norm
- LightGCN- L_1 -R: Chỉ chuẩn hóa phải với L_1 -norm
- LightGCN- L_1 : Chuẩn hóa đối xứng nhưng dùng L_1 -norm
- LightGCN-L: Chỉ chuẩn hóa trái với L_2 -norm
- LightGCN-R: Chỉ chuẩn hóa phải với L_2 -norm

Ta có thể nhận thấy từ bảng so sánh trên:

- LightGCN chuẩn hóa đối xứng (L_2 -norm hai bên) luôn cho kết quả tốt nhất trên tất cả các dataset.
- Nếu chỉ chuẩn hóa một phía thì hiệu suất kém hơn rõ rệt

3.8.3 Phân tích độ mượt (Embedding Smoothness) trong LightGCN

Smoothness (độ mượt) của embedding: Để đo mức độ giống nhau giữa embedding của các user (hoặc item) có liên quan/đồng nhất các tương tác.

- Embedding "mượt" hơn tức là các user tương tự nhau (ví dụ từng thích những item giống nhau) sẽ có embedding gần nhau trong không gian vector.
- Độ mượt này giúp hệ thống gợi ý dự đoán chính xác hơn, vì có thể nhận diện tốt các nhóm người dùng có cùng sở thích/tương tác.
- Độ mượt thấp hơn $\rightarrow$ embedding của các user liên quan gần nhau hơn, tốt cho collaborative filtering

Công thức đo độ mượt:

$$S_U = \sum_{u=1}^M \sum_{v=1}^M c_{v \rightarrow u} \left(\frac{\mathbf{e}_u}{|\mathbf{e}_u|^2} - \frac{\mathbf{e}_v}{|\mathbf{e}_v|^2} \right)^2$$

Dataset	Gowalla	Yelp2018	Amazon-book
Smoothness of User Embeddings			
MF	15449.3	16258.2	38034.2
LightGCN-single	12872.7	10091.7	32191.1
Smoothness of Item Embeddings			
MF	12106.7	16632.1	28307.9
LightGCN-single	5829.0	6459.8	16866.0

Figure 9: Độ mất mát mượt mà của các nhúng được học bởi LightGCN và MF

Ta có thể rút ra nhận xét như sau:

- LightGCN-single cho độ mượt tốt hơn so với MF

3.9 Hyper-parameter

Ngoài learning rate, hệ số L2 regularization (λ) là thông số cần quan tâm nhất khi áp dụng LightGCN lên dataset mới.

LightGCN khá ít nhạy cảm với λ (không dễ bị overfit khi λ thấp), vì:

- Chỉ có embedding ban đầu là tham số học (không có ma trận W, không nonlinear), nên ít nguy cơ overfitting như các model phức tạp khác.
- Ngay cả khi $\lambda = 0$, LightGCN vẫn hiệu quả hơn NGCF (NGCF còn phải dùng thêm dropout).

- Khi triển khai LightGCN, chỉ cần tuning λ trong khoảng $1e-4$ đến $1e-3$. Mô hình này rất khó overfit, không cần regularization mạnh như các GCN truyền thống, và thường không cần thêm dropout. Đặt λ quá lớn sẽ làm giảm chất lượng gợi ý.

3.10 Tóm lại

LightGCN chứng minh các thiết kế phức tạp của GCN truyền thống là không cần thiết cho collaborative filtering (CF):

- Bỏ các bước biến đổi đặc trưng (feature transformation) và phi tuyến (nonlinear activation), giúp mô hình đơn giản, dễ train, ít overfitting.
- Kết hợp nhiều layer (layer combination) để lấy embedding cuối cùng, vừa tận dụng được self-connection, vừa kiểm soát oversmoothing (embedding bị mờ nhạt khi truyền nhiều lớp).

LightGCN vượt trội hơn các mô hình SOTA hiện tại:

- Đơn giản hơn, dễ triển khai
- Tổng quát hóa tốt hơn, hiệu quả hơn trên các bộ dữ liệu lớn, phức tạp

3.11 Ưu điểm và nhược điểm

3.11.1 Ưu điểm

- Đơn giản, dễ triển khai:
 - Bỏ qua bước biến đổi đặc trưng (feature transformation) và phi tuyến (nonlinear activation), chỉ giữ lại bước lan truyền (aggregation) rất “nhẹ”.
 - Code ngắn gọn, dễ tối ưu, dễ tune, phù hợp mọi framework.
- Dễ train, khó overfitting:
 - Chỉ cần học embedding gốc cho user/item, không có nhiều tham số phức tạp.
 - Dễ hội tụ, ít khi cần regularization mạnh, không cần dropout như các GCN khác.

- Hiệu quả cao trên nhiều tập dữ liệu thực tế
 - Thường outperform các GCN phức tạp (NGCF, GCMC, GAT, ...) trên recommendation (đặc biệt dữ liệu thưa).
 - Rất mạnh với bài toán collaborative filtering khi không có nhiều feature phụ.
- Khả năng tổng quát hóa tốt
 - Layer combination (kết hợp multi-layer embedding) giúp tránh over-smoothing khi số layer lớn.

3.11.2 Nhược điểm

- Không tận dụng được tốt các feature phức tạp
 - Nếu bài toán có nhiều thông tin content, profile, text/image... thì các mô hình hybrid (content-based GNN, DeepCF, ...), hoặc GCN có nonlinear sẽ tận dụng tốt hơn.
- Không modeling được quan hệ phức tạp
 - Chỉ lan truyền trên đồ thị user-item, không trực tiếp model các mối quan hệ đa chiều (multi-relational, heterogeneous graph) như user-user, item-item, item-category, ...
 - Đồ thị phức tạp thì cần nhiều bổ sung hơn.
- Yếu khi dữ liệu cực lạnh/cold-start không có feature
- Embedding kích thước lớn dễ tốn RAM khi user/item nhiều

4 Giới thiệu về dataset

Dataset Retailrocket recommender system dataset gồm có bốn file csv:

- File events.csv: Chứa log toàn bộ tương tác (hành vi) của user với item trong suốt 4.5 tháng. Dataset có 2756101 event gồm 2664312 views, 69332 add to carts và 22457 transactions bởi 1407580 visitors riêng biệt
 - timestamp: Thời điểm xảy ra sự kiện (đơn vị: milliseconds, Unix timestamp)

- visitorid: ID của user (người dùng, ẩn danh)
 - event: Loại hành vi (view, addtocart, transaction)
 - itemid: ID của sản phẩm
 - transactionid: Mã hóa đơn (chỉ có nếu event là 'transaction')
- File itemproperties.csv: Chứa tập các thuộc tính (property) của item – ví dụ: giá, tình trạng còn hàng, ngành hàng, thương hiệu... Mỗi thuộc tính có thể thay đổi theo thời gian, nên file lưu cả timestamp cho từng giá trị. File này sẽ chia thành hai file con.
 - timestamp: Thời điểm thuộc tính đó có giá trị này.
 - itemid: ID sản phẩm
 - property: Thuộc tính (giá, categoryid, available, tên sản phẩm, brand, ...)
 - value: Giá trị của thuộc tính (giá trị số, mã ngành, còn hàng hay không, v.v.)
 - File categorytree.csv: Mô tả cấu trúc phân cấp (hierarchy) của ngành hàng (category) các sản phẩm. Mỗi dòng là một cặp categoryid, parentid – thể hiện mối quan hệ cha-con giữa các ngành hàng.
 - categoryid: ID ngành hàng (con)
 - parentid: ID ngành hàng cha (nếu trống nghĩa là gốc)

5 Hiện thực LightGCN

5.1 Load dữ liệu

Ở đây do LightGCN bản chất sử dụng mối quan hệ giữa user-item nên ta chỉ sử dụng file events.csv

```
1 events=pd.read_csv('events.csv')
2 events.head(3)
```

Kết quả:

	timestamp	visitorid	event	itemid	transactionid
0	1433221332117	257597	view	355908	NaN
1	1433224214164	992329	view	248676	NaN
2	1433221999827	111016	view	318965	NaN

Figure 10: Dữ liệu events

5.2 Kiểm tra dữ liệu

5.2.1 Kích thước dữ liệu

```
1 def check_info(events):  
2     print('The size of events:', events.shape)  
3     print('The number of itemid unique:', events['  
4         itemid'].nunique())  
5     print('The number of visitorid unique:', events['  
6         visitorid'].nunique())  
7     print(events['event'].value_counts())  
8     check_info(events)
```

Kết quả:

```
The size of events: (2756101, 5)  
The number of itemid unique: 235061  
The number of visitorid unique: 1407580  
event  
view          2664312  
addtocart      69332  
transaction    22457  
Name: count, dtype: int64
```

Figure 11: Thông tin dữ liệu eventsevents

Số tương tác của dữ liệu có 2756101. Trong đó, có 2664312 số lượng event "view", 69332 số lượng event "addtocart", 22457 số lượng event "transaction". Bên cạnh đó, có 235061 các item riêng biệt, và 1407580 người dùng.

5.2.2 Kiểm tra dữ liệu null

```
1 print(events.isnull().sum())
```

Kết quả:

```
timestamp      0
visitorid      0
event          0
itemid         0
transactionid   2733644
dtype: int64
```

Figure 12: Kết quả dữ liệu null ở mỗi cột

Không có dữ liệu null trong cả 5 cột.

5.2.3 Xử lý các dữ liệu trùng lặp

Ta tiến hành xóa đi các dữ liệu trùng lặp nhau.

```
1 events.drop_duplicates(inplace=True)
2 check_info(events)
```

Kết quả:

```
The size of events: (2755641, 5)
The number of itemid unique: 235061
The number of visitorid unique: 1407580
event
view      2664218
addtocart   68966
transaction 22457
Name: count, dtype: int64
```

Figure 13: Thông tin của dữ liệu sau khi xóa trùng lặp

Nhận xét:

5.2.4 Tần suất của các giá trị trong cột event

```
1 print(events['event'].value_counts())
2 print("\nPercentage of events:")
3 print(events['event'].value_counts
```

```
4 (normalize=True) * 100)
```

Kết quả:

```
event
view      2664312
addtocart   69332
transaction  22457
Name: count, dtype: int64

Percentage of events:
event
view      96.669607
addtocart   2.515583
transaction  0.814810
Name: proportion, dtype: float64
```

Figure 14: Tần suất của từng loại event

```
1 def distribution_of_the_events(events):
2     plt.figure(figsize=(8,5))
3     eventsCounting=events['event'].value_counts()
4     sns.barplot(x=eventsCounting.index,y=eventsCounting
5                 .values, color='#c94169')
6     plt.title('The ditribution of the event')
7     plt.xlabel('Type of event')
8     plt.ylabel(f'Number of events')
9     plt.show()
distribution_of_the_events(events)
```

Kết quả:

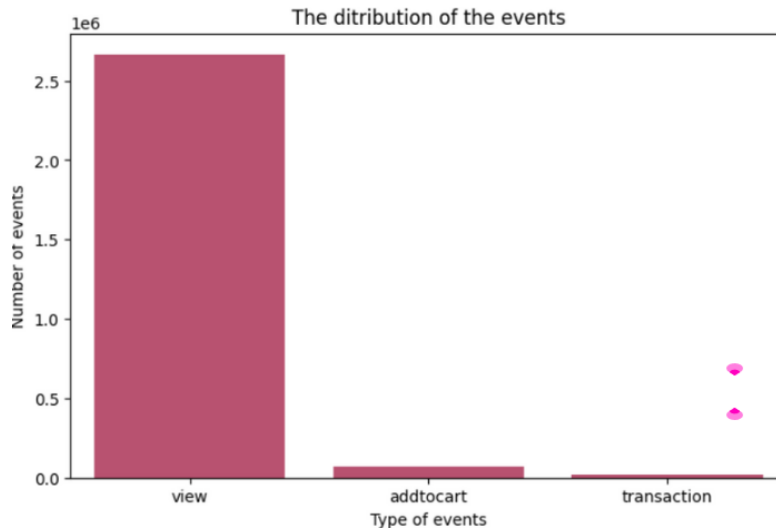


Figure 15: Biểu đồ tần suất các loại event

Nhận xét: Ta thấy có sự mất cân bằng rất lớn trong dữ liệu. Trong khi sự kiện "view" phần lớn hơn 96%, sự kiện "addtocart" chiếm 2.52% và sự kiện "transaction" chỉ chiếm 0.81%. Do vậy ta cần phải điều chỉnh sự mất cân bằng này lại.

5.2.5 Kiểm tra độ dày của dữ liệu

Ta tiến hành kiểm tra độ dày của dataset. Độ dày của dataset chính là độ thưa của ma trận tương tác (user-item interactions matrix) : Công thức độ dày :

$$Sparsity = \frac{|D|}{N_{user} \times N_{item}}$$

Trong đó:

- $|D|$: số lượng tương tác thực tế (cặp user-item có dữ liệu)
- N_{user} : Số lượng người dùng duy nhất
- N_{item} : Số lượng item duy nhất

```
1 def Calculate_sparsity_density(events):
2     sparsity = 1 - len(events) / (events['visitorid']
    ].nunique() * events['itemid'].nunique())
```

```
3 density= len(events) / (events['visitorid'].  
4 nunique() * events['itemid'].nunique())  
5 print(f"Sparsity: {sparsity:.4f}")  
6 print(f"Density: {density:.4f}")  
Calculate_sparsity_density(events)
```

Kết quả:

- Sparsity: 1.0000
- Density: 0.0000

Nhận xét: Dữ liệu có độ dày gần như bằng 0, chứng tỏ dữ liệu có độ thưa rất lớn. Đây là tình trạng phổ biến trong thực tế, nhất là với các hệ thống lớn.

5.2.6 Đồ thị thể hiện sự phân phối

Ta vẽ đồ thị thể hiện sự phân phối của số lượng itemid theo số lượng event.

```
1 def distribution(events, x):  
2     if x=='itemid' or x=='visitorid' or x=='property':  
3         counts = events[x].value_counts()  
4         plt.figure(figsize=(10, 6))  
5         plt.hist(counts.values, bins=50, color='pink')  
6         if x == 'itemid':  
7             plt.xlabel('Event with itemid')  
8             plt.ylabel('Number of itemid')  
9             plt.title('Distribution of itemid count by  
10                event')  
11         elif x=='visitorid':  
12             plt.xlabel('Event with visitorid')  
13             plt.ylabel('Number of visitorid')  
14             plt.title('Distribution of visitorid count by  
15                event')  
16         plt.yscale('log')  
17         plt.show()  
18     else:  
19         print('Error')  
20     distribution(events, 'itemid')
```

Kết quả:

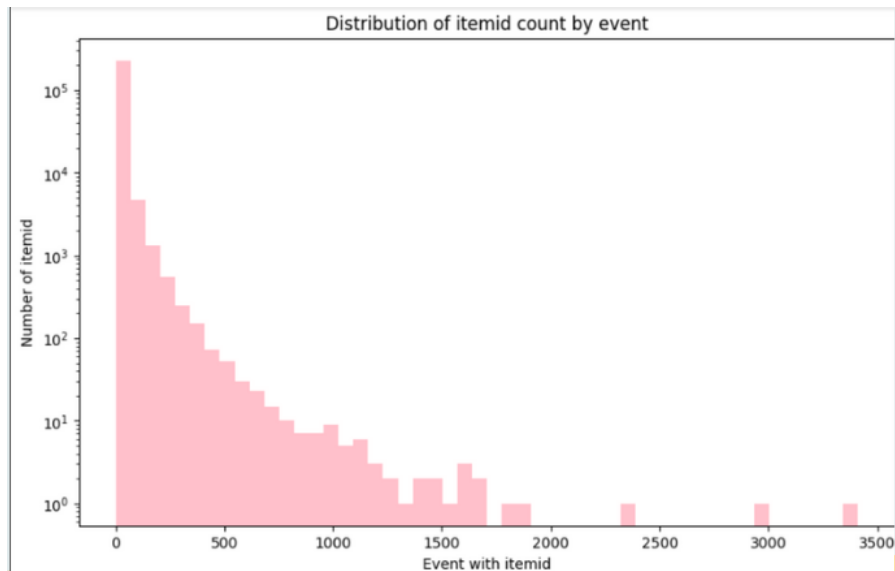


Figure 16: Biểu đồ thể sự phân phối số lượng itemid theo số lượng event

Nhận xét: Đồ thị thể hiện phân phối của số lượng itemid theo số lượng event. Nhìn vào đồ thị ta thấy được đồ thị giảm dần về phía phải (dữ liệu bị lệch phải). Điều này cho thấy sự phân bố không đều của dữ liệu, số lượng itemid có nhiều event chiếm rất ít trong dataset. Trong khi đó, số lượng itemid trong tập dữ liệu có nhiều event lại chiếm rất ít. Điều này thể hiện được độ thưa của dữ liệu. Do đó, chúng ta cần có biện pháp sampling để tăng độ dày của dữ liệu giúp cho mô hình học tốt hơn.

```
1 def top10_low(events, x):
2     if x=='itemid' or x=='visitorid':
3         counts = events[x].value_counts()
4         item_freq_count = counts.value_counts().
5             sort_index()
6         top10_low_freq = item_freq_count.head(10)
7         plt.figure(figsize=(8, 5))
8         bars = plt.bar(top10_low_freq.index.astype(str),
9             top10_low_freq.values, color='#c94169')
10        if x == 'itemid':
11            plt.xlabel('Event with itemid')
12            plt.ylabel('Number of itemid')
13            plt.title('Top 10 low distribution of itemid
14                count by event')
15        else:
```

```
13     plt.xlabel('Event with visitorid')
14     plt.ylabel('Number of visitorid')
15     plt.title('Top 10 low distribution of visitorid
16               count by event')
17     plt.tight_layout()
18
19     total_items = counts.shape[0]
20     for i, bar in enumerate(bars):
21         pct = 100 * top10_low_freq.values[i] /
22             total_items
23         plt.text(bar.get_x() + bar.get_width()/2, bar
24                 .get_height(), f'{pct:.1f}%', ha='center',
25                 va='bottom', fontsize=9)
26
27     plt.show()
28 else:
29     print('Error')
30 top10_low(events, 'itemid')
```

Kết quả:

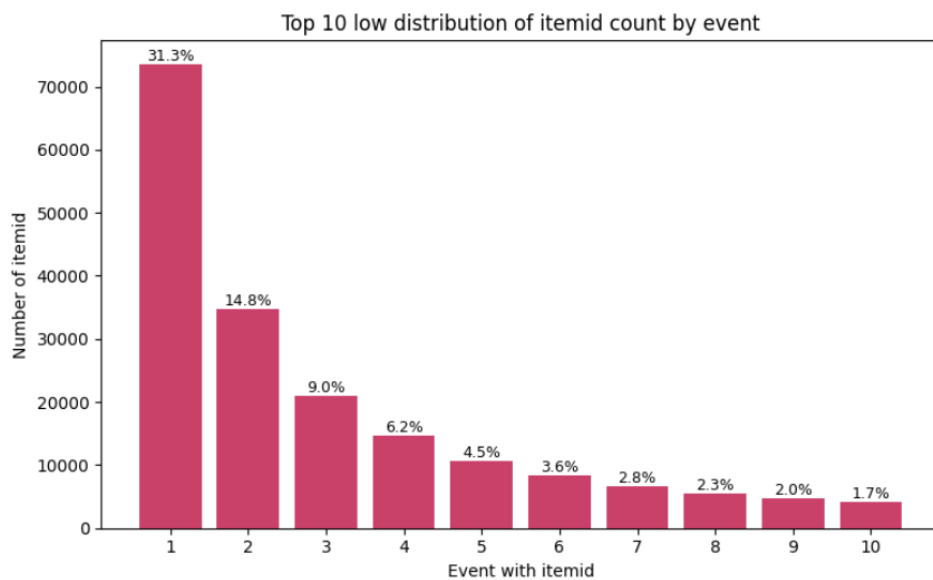


Figure 17: Biểu đồ top 10 item có số lượng event thấp nhất

Nhận xét: Ta thấy được số lượng itemid có số lượng event dưới 4 chiếm đến 55,1% số lượng itemid riêng biệt. Điều này phản ánh đặc trưng “đuôi

dài” trong thương mại điện tử, khi số ít sản phẩm “hot” nhận được nhiều click, còn đại đa số sản phẩm “lạnh” hầu như không được chú ý. Do vậy, để mô hình LightGCN hoạt động tốt thì cần có biện pháp điều chỉnh, mô hình ưu ái các item phổ biến và bỏ qua những mặt hàng ít dữ liệu. Ta vẽ đồ thị thể hiện sự phân phối của số lượng visitorid theo số lượng event.

```
1 distribution(events, 'visitorid')
```

Kết quả:

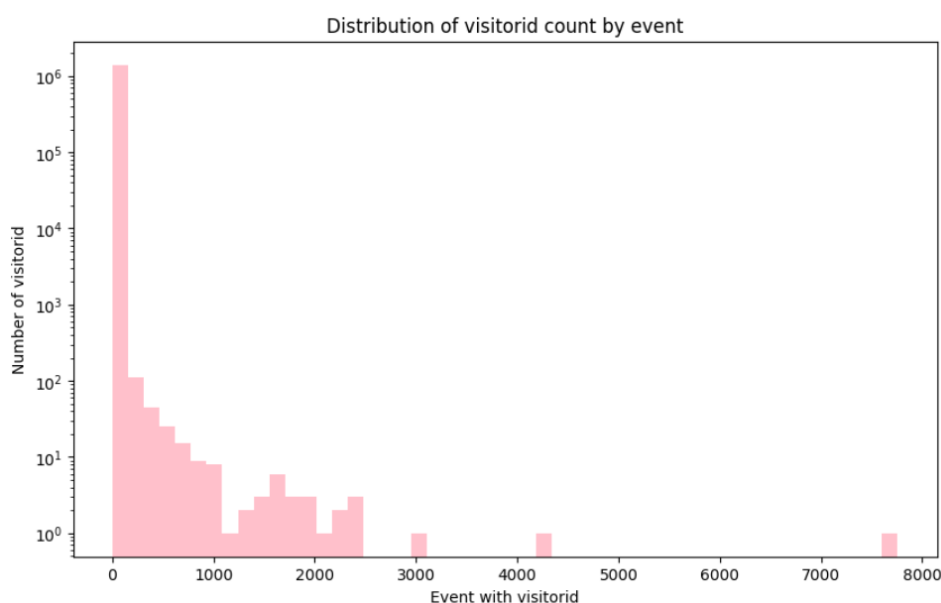


Figure 18: Biểu đồ sự phân phối số lượng visitorid theo số lượng event

Nhận xét: Giống với phân phối của itemid, biểu đồ cực lệch phải. Số lượng người dùng có số event thấp chiếm rất cao, chỉ có một nhóm nhỏ có nhiều event. Điều này làm mất cân bằng dữ liệu, và làm độ thưa dữ liệu rất cao (độ dày rất nhỏ).

```
1 top10_low(events, 'visitorid')
```

Kết quả:

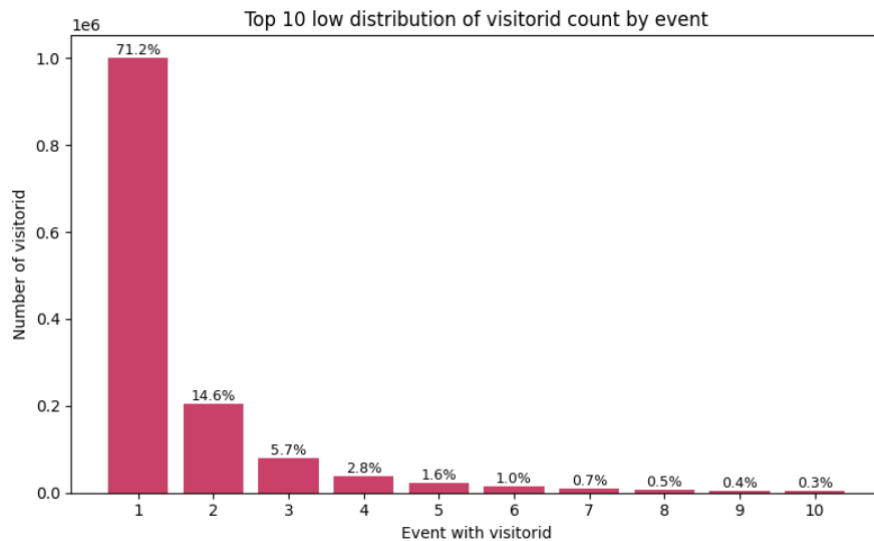


Figure 19: Biểu đồ top 10 visitorid có số lượng event thấp nhất

Nhận xét: Người dùng có số lượng event dưới 3 chiếm đến 85.8%. Điều này cho thấy dữ liệu cực kỳ thừa. Kết quả này phản ánh hiện tượng “long-tail” rõ nét—đa số người dùng rất hời hợt, để lại dữ liệu cực ít, còn nhóm người dùng “nhiệt” chỉ chiếm tỉ lệ nhỏ nhưng tạo phần lớn tín hiệu hành vi. Tóm lại, sau khi xem xét các phân phối trên thì ta thấy độ thừa, và cold-start rất nặng.

5.2.7 Thống kê mô tả

Ta tính thống kê mô tả của số lượng event theo itemid.

```
1 def statistic(event, x):
2     if x=='itemid' or x=='visitorid':
3         counts = event[x].value_counts()
4         print(counts.describe())
5     else:
6         print('Error')
7     statistic(events, 'itemid')
```

Kết quả:

	count
count	235061.000000
mean	11.723089
std	37.024810
min	1.000000
25%	1.000000
50%	3.000000
75%	9.000000
max	3412.000000

Figure 20: Kết quả thống kê số lượng event theo itemid

Nhận xét: Có 235061 item trong tệp dữ liệu. Trong đó, trung vị chỉ có 3 (tức là một nửa item có số lượng event tối đa là 3), nhưng trung bình tới 11.7, độ lệch chuẩn 37.02 (xấp xỉ gần ba lần trung bình) chứng tỏ có một đuôi rất dài có vài item có rất nhiều event. Bên cạnh đó, ta thấy giá trị max là 3412 trong khi 75% có tối đa là 9 event, chứng tỏ ngoại lệ chi phối phần lớn tổng lượt tương tác. Tóm lại, hậu quả là ma trận user-item cực thưa. Ta tính thống kê mô tả của số lượng event theo visitorid.

```
1 statistic(events, 'visitorid')
```

Kết quả:

count	
count	1.407580e+06
mean	1.957715e+00
std	1.257981e+01
min	1.000000e+00
25%	1.000000e+00
50%	1.000000e+00
75%	2.000000e+00
max	7.757000e+03

Figure 21: Kết quả thống kê số lượng event theo visitorid

Nhận xét: Trong hơn 1.4 triệu người dùng thì trung vị là chỉ có 1, 75 % là có nhiều nhất là 2 event. Điều này cho thấy đa số thông tin về người dùng bị thiếu rất nhiều. Tuy trung bình là 1,96 event, độ lệch chuẩn tới 12,6 cho thấy đuôi phải rất dài. Vài tài khoản “siêu hoạt động” (tối đa 7 757 event) kéo mean lên nhưng chiếm tỉ lệ cực nhỏ.

5.3 Tiền xử lý dữ liệu

Để mô hình hoạt động tốt hơn, chúng ta cần phải giảm độ thừa của dữ liệu và sự mất cân bằng quá lớn giữa các sự kiện "view", "transaction", "addtocart". Ta tiến hành giữ lại toàn bộ event "addtocart", "transaction" và chỉ lấy 10% sự kiện "view".

```
1 addtocarts = events[events['event'] == 'addtocart']
2 transactions = events[events['event'] == 'transaction']
3
4 views = events[events['event'] == 'view'].sample(frac=0.1, random_state=42)
5 final_events = pd.concat([views, addtocarts, transactions]).reset_index(drop=True)
6 check_info(final_events)
7 print ( " \ nPercentage of events : " )
```

```
7 print ( final_events['event'].value_counts( normalize  
= True ) * 100)
```

Kết quả:

```
The size of events: (357845, 5)
The number of itemid unique: 94202
The number of visitorid unique: 242807
event
view          266422
addtocart      68966
transaction    22457
Name: count, dtype: int64
\ nPercentage of events :
event
view          74.451788
addtocart     19.272590
transaction    6.275622
Name: proportion, dtype: float64
```

Figure 22: Kết quả sau khi cắt giảm event "view"

Nhận xét: Sau khi cắt thì "view" còn 266422 (chiếm 74.45%), "addtocart" và "transaction" lần lượt giữ 19.27% và 6.28%. Kích thước của dữ liệu sau khi cắt, dữ liệu còn 357845 tương tác. Trong đó, số lượng item còn 94202, số lượng người dùng còn 242807.

Tiếp theo ta tiến hành loại bỏ user/item có ít tương tác để giảm nhiễu. Hàm này sẽ tiến hành lọc đến khi nào mà cả người dùng và item đều có số event của mỗi người dùng ít nhất bằng k_{user} , số event của mỗi item ít nhất bằng k_{item} . Ở đây, ta chọn loại bỏ các user nhỏ hơn `user_counts.mean()`, các item có số event nhỏ hơn 2

```
1 from typing_extensions import final
2 def filter_k_core(data, k_user, k_item):
3     print("Start Filtering K-Core...")
4     while True:
5         user_counts = data['visitorid'].value_counts()
6         item_counts = data['itemid'].value_counts()
7         weak_users = user_counts[user_counts < k_user].index
8         weak_items = item_counts[item_counts < k_item].index
```

```
9         initial_rows = len(data)
10         users_before = data['visitorid'].nunique()
11         items_before = data['itemid'].nunique()
12
13         data = data[~data['visitorid'].isin(
14             weak_users)]
15         data = data[~data['itemid'].isin(weak_items)]
16
17         if len(weak_users) == 0 and len(weak_items)
18             == 0:
19             print("Finish filtering K-Core.")
20             break
21
22         print(f"Remove {users_before - data['
23             visitorid'].nunique()} users, {
24             items_before - data['itemid'].nunique()}
25             items, {initial_rows - len(data)} rows.")
26
27     return data
28
29     print("\nThe size of dataset before filtering:")
30     check_info(final_events)
31     user_counts = final_events['visitorid'].value_counts
32         ()
33     itemid_counts = final_events['itemid'].value_counts()
34     filtered_events = filter_k_core(final_events.copy(),
35         k_user= 2, k_item=itemid_counts.mean())
36     print("The size of dataset after filtering:")
37     check_info(filtered_events)
```

Kết quả:

```
The size of dataset before filtering:
The size of events: (357845, 5)
The number of itemid unique: 94202
The number of visitorid unique: 242807
event
view          266422
addtocart     68966
transaction   22457
Name: count, dtype: int64
Start Filtering K-Core...
Remove 206127 users, 74005 items, 231296 rows.
Remove 7343 users, 8431 items, 20591 rows.
Remove 3077 users, 534 items, 4338 rows.
Remove 443 users, 404 items, 1434 rows.
Remove 236 users, 45 items, 355 rows.
Remove 38 users, 35 items, 129 rows.
Remove 15 users, 2 items, 20 rows.
Remove 3 users, 3 items, 10 rows.
Remove 1 users, 0 items, 1 rows.
Finish filtering K-Core.
The size of dataset after filtering:
The size of events: (99671, 5)
The number of itemid unique: 10743
The number of visitorid unique: 25524
event
view          43075
addtocart     38865
transaction   17731
Name: count, dtype: int64
```

Figure 23: Kết quả sau khi lọc

Nhận xét: Sau khi lọc thì kích thước dữ liệu còn 99671 dòng, số lượng người dùng còn 25524, số lượng item còn 10743. Việc sample dữ liệu là điều cần thiết do dữ liệu ban đầu quá thừa, và có hiện tượng "cold-start". Do vậy để mô hình LightGCN có thể học tốt thì đây là điều cần thiết phải làm.

Sau khi lọc, thì phân phối của các event như sau:
Kết quả:

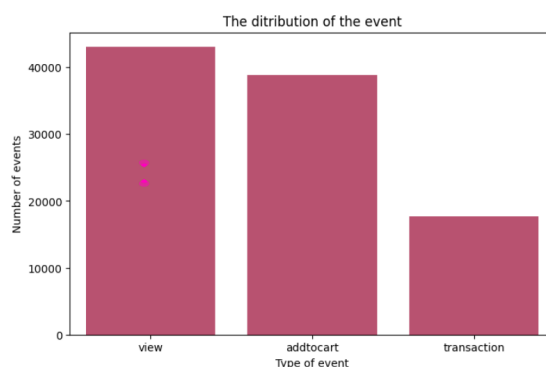
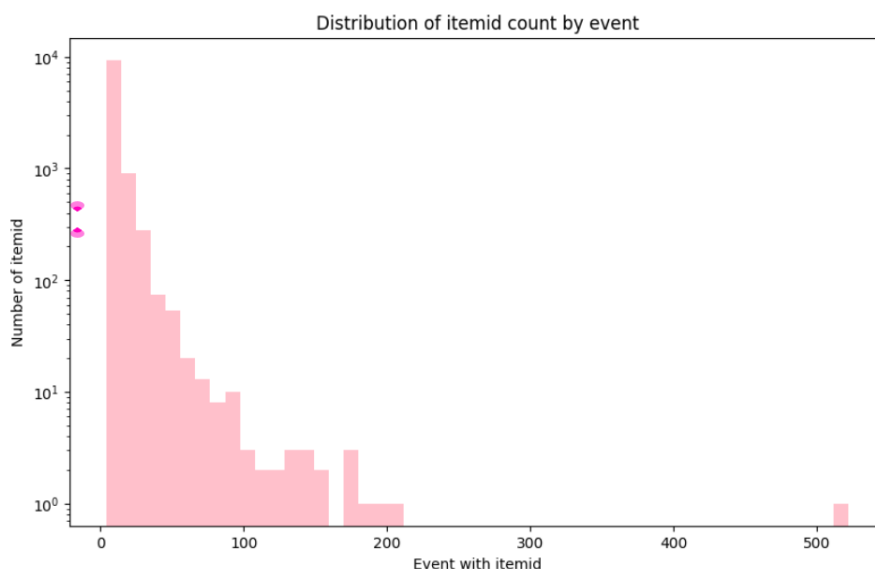


Figure 24: Biểu đồ tần suất các loại event sau khi xử lý

Nhận xét: Ta thấy dữ liệu đã ít đi sự mất cân bằng.

Độ dày của dữ liệu sau khi xử lý dữ liệu là : 0.0004 (đã tăng lên so với ban đầu là 0.0000)



Nhận xét: Sau khi loại bỏ những user, item quá ít tương tác, thì dataset đã trở nên dày và cân đối hơn so với ban đầu. Đuôi phải co ngắn làm giảm thiên lệch về một số ít item “hot”. Nhờ vậy, LightGCN chỉ cần 2–3 layer để đạt hiệu quả, tốc độ huấn luyện nhanh hơn và nguy cơ over-smoothing thấp.

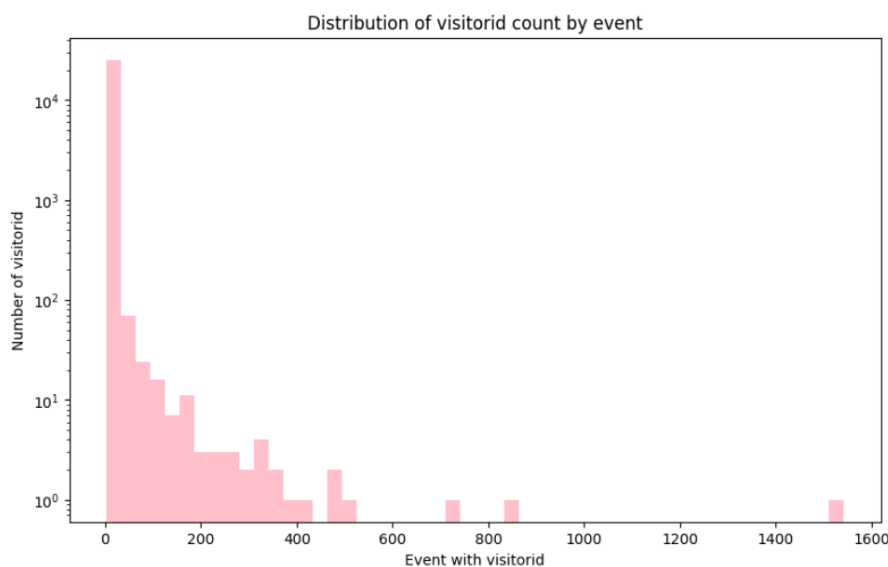


Figure 27: Biểu đồ sự phân phối số lượng theo số lượng event

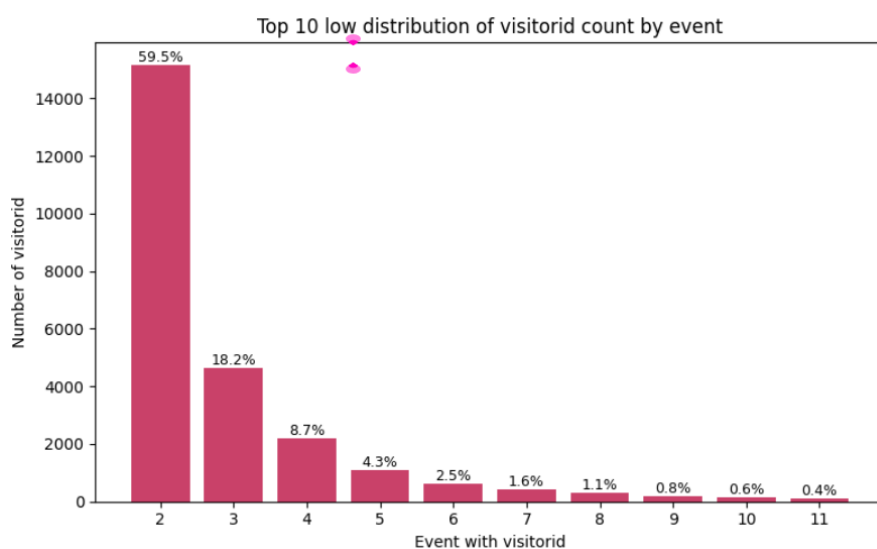


Figure 28: Biểu đồ top 10 visitorid có số lượng event thấp nhất

Nhận xét: Sau khi loại bỏ đi người dùng có một event thì phân phối user-event đã “dịu” hơn nhưng vẫn giữ đặc trưng long-tail. Có 59.5% người dùng chỉ có 2, 18.2% chỉ có 3 event, 8.7% có 4 event, tức là hiện tại có khoảng 86.4% người dùng có từ 2-4 event. So với trước đây, 71.2% chỉ có 1 event thì như vậy đã tăng lên rất nhiều, nhưng cold-start vẫn là một thách thức vì phần lớn người dùng cũng chỉ có 2 event. Với mật độ mới, 2-3 layer trong LightGCN là đủ, tăng thêm layer sẽ mang lại ít lợi ích vì đa số user chỉ có vài bước hàng xóm.

5.4 Hiện thực LightGCN thuần

5.4.1 Chuẩn bị dữ liệu

Đầu tiên, ta tiến hành gán trọng số cho từng event "view", "addtocart", "transaction". Ở đây ta tiến hành gán trọng số cho từng event nhằm để mô hình hiểu được mức độ quan trọng của từng event. "View" là tín hiệu thể hiện người dùng quan tâm nhưng khá yếu, có thể người dùng chỉ xem lướt qua. Nhưng "addtocart" là tín hiệu mạnh hơn, thể hiện người dùng có sự quan tâm mạnh mẽ hơn. "transaction" là tín hiệu mạnh nhất, thể hiện được sự quan tâm, yêu thích cao nhất của người dùng đối với sản phẩm.

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from sklearn.preprocessing import LabelEncoder
6 event_weight = {
7     'view': 1.0,
8     'addtocart': 3.0,
9     'transaction': 5.0
10 }
```

Ta tiến hành tạo một dataframe "interactions" chứa ba cột "visitorid", "itemid", "event", xóa đi những dữ liệu trùng lặp (ví dụ như một itemid mà visitorid đó xem đi xem lại nhiều lần thì chỉ giữ lại một lần sự kiện đó thôi). Vì nếu không làm vậy sẽ bị đếm nhiều lần một cách không công bằng. Tiếp đó, ta gán những trọng số cho các loại event trong "interactions".

```
1 interactions = filtered_events[['visitorid', 'itemid',
2                                'event']]
3 interactions = interactions.drop_duplicates()
```

```
3 interactions['weight'] = interactions['event'].map(  
    event_weight)
```

Ta tạo ra hai khối để mã hóa cho người dùng và item. Sau đó, phương thức *fit_transform* sẽ quét qua toàn bộ cột của `interactions['visitorid']` và `interactions['itemid']` để học và xác định được những giá trị riêng biệt "visitorid", "itemid". Sau đó, nó sẽ tạo ra một bảng ánh xạ nội bộ để gán cho mỗi giá trị đó một số nguyên liên tiếp bắt đầu từ 0. Kết quả sẽ được lưu vào cột mới "user", "item".

```
1 user_encoder = LabelEncoder()  
2 item_encoder = LabelEncoder()  
3 interactions['user'] = user_encoder.fit_transform(  
    interactions['visitorid'])  
4 interactions['item'] = item_encoder.fit_transform(  
    interactions['itemid'])
```

5.4.2 Chia train/test/validation

Tiếp theo, ta tiến hành chia tập dữ liệu thành train/test/validation cho từng người dùng. Đầu tiên, ta sắp xếp dữ liệu của từng người dùng theo "timestamp". Ở test dataset thì chúng ta lấy hành động cuối của người dùng có từ hai event trở lên. Sau đó, chúng ta loại bỏ những index đó ra khỏi tập dữ liệu. Tiếp theo, tập validation thì sau khi lọc vậy thì ta lấy những người dùng có từ 2 event trở lên làm tập validation.

```
1 interactions = interactions.sort_values(['user', '  
    timestamp'])  
2  
3 print("Start splitting with special handling for  
    users with 1 event...")  
4 user_event_counts = interactions['user'].value_counts(  
    )  
5  
6 users_with_1_event = user_event_counts[  
    user_event_counts == 1].index  
7 users_with_2_events = user_event_counts[  
    user_event_counts == 2].index  
8 users_with_3plus_events = user_event_counts[  
    user_event_counts >= 3].index  
9
```

```
10     print(f"Users with 1 event: {len(users_with_1_event)}")
11     print(f"Users with 2 events: {len(users_with_2_events)}")
12     print(f"Users with 3+ events: {len(users_with_3plus_events)}")
13
14     # Only users with 2+ events can have test data
15     if len(users_with_2_events) + len(users_with_3plus_events) > 0:
16         test_idx = interactions[interactions['user'].isin(
17             (user_event_counts[user_event_counts >= 2].index))].groupby('user').tail(1).index
18         test_interactions = interactions.loc[test_idx]
19     else:
20         test_interactions = pd.DataFrame(columns=interactions.columns)
21         test_idx = pd.Index([])
22
23     # Remaining data
24     remaining_after_test = interactions.drop(test_idx)
25
26     # --- 2. GET VALIDATION SET ---
27     # Users with 2+ events (after subtracting test) can have validation data
28     remaining_user_counts = remaining_after_test['user'].value_counts()
29     users_for_val = remaining_user_counts[remaining_user_counts >= 2].index
30
31     if len(users_for_val) > 0:
32         val_idx = remaining_after_test[remaining_after_test['user'].isin(
33             users_for_val)].groupby('user').tail(1).index
34         val_interactions = remaining_after_test.loc[val_idx]
35     else:
36         val_interactions = pd.DataFrame(columns=interactions.columns)
37         val_idx = pd.Index([])
```

```
37 # --- 3. GET TRAIN SET ---
38 # All remaining data (including users with 1 event)
   will go to train
39 train_interactions = remaining_after_test.drop(
   val_idx)
40
41 print(f"\nData split results:")
42 print(f"Train: {len(train_interactions)} interactions
   , {train_interactions['user'].nunique()} users")
43 print(f"Validation: {len(val_interactions)}
   interactions, {val_interactions['user'].nunique()}
   users")
44 print(f"Test: {len(test_interactions)} interactions,
   {test_interactions['user'].nunique()} users")
45
46 # Users with 1 event will only appear in train
47 users_1_event_in_train = set(users_with_1_event) &
   set(train_interactions['user'].unique())
48 print(f"Users with 1 event in train: {len(
   users_1_event_in_train)}")
```

Kết quả sau khi chia:

```
Start splitting with special handling for users with 1 event...
Users with 1 event: 3530
Users with 2 events: 14019
Users with 3+ events: 7975

Data split results:
Train: 54484 interactions, 25524 users
Validation: 7975 interactions, 7975 users
Test: 21994 interactions, 21994 users
Users with 1 event in train: 3530
```

5.4.3 Tạo edge index và edge weight cho train

Ta biến dữ liệu dạng bảng sang thành cấu trúc đồ thị để làm đầu vào cho mô hình LightGCN. Trong đó, các node là người dùng và item, cạnh là sự thể hiện xem mỗi liên hệ giữa người dùng và item. Mỗi cạnh sẽ có một trọng số khác nhau tùy thuộc vào loại event. Các mô hình đồ thị như LightGCN hoạt động tốt nhất trên đồ thị vô hướng (undirected graph), nghĩa là thông tin có thể lan truyền theo cả hai chiều: từ user $\rightarrow$ item và từ item $\rightarrow$ user. `torch.cat(...)` sẽ ghép hai tập đường nối này lại với nhau. Sau đó, ta tiến hành nhân đôi trọng số cho các đường nối hai chiều.

```
1 user_indices = torch.tensor(train_interactions['user']
2                               ].values, dtype=torch.long)
3 item_indices = torch.tensor(train_interactions['item']
4                               ].values, dtype=torch.long) + num_users
5 edge_weight = torch.tensor(train_interactions['weight']
6                               ].values, dtype=torch.float)
7 edge_index = torch.cat([
8     torch.stack([user_indices, item_indices]),
9     torch.stack([item_indices, user_indices])
10 ], dim=1)
11 edge_weight = torch.cat([edge_weight, edge_weight])
```

5.4.4 LightGCN

Ta hiện thực mô hình LightGCN

```
1 class LightGCN(nn.Module):
2     def __init__(self, num_users, num_items,
3                   embedding_dim=128, n_layers=2):
4         super().__init__()
5         self.num_users = num_users
6         self.num_items = num_items
7         self.embedding_dim = embedding_dim
8         self.n_layers = n_layers
9         self.user_embedding = nn.Embedding(num_users,
10                                             embedding_dim)
11        self.item_embedding = nn.Embedding(num_items,
12                                            embedding_dim)
13        nn.init.xavier_uniform_(self.user_embedding.
14                                weight)
15        nn.init.xavier_uniform_(self.item_embedding.
16                                weight)
17        self.alpha = nn.Parameter(torch.ones(self.
18                                           n_layers + 1) / (self.n_layers + 1))
19
20    def forward(self, edge_index, edge_weight):
21        all_embeddings = torch.cat([self.
22                                    user_embedding.weight, self.item_embedding.
23                                    weight])
24        embeddings_list = [all_embeddings]
25        for _ in range(self.n_layers):
```

```
18         all_embeddings = self.propagate(  
19             edge_index, edge_weight,  
20             all_embeddings)  
21         embeddings_list.append(all_embeddings)  
22  
23         import torch.nn.functional as F  
24         weights = F.softmax(self.alpha, dim=0) # [  
25             n_layers+1]  
26         stacked = torch.stack(embeddings_list, dim=1)  
27         # [num_nodes, n_layers+1, emb_dim]  
28         final_embedding = (stacked * weights[None, :,  
29             None]).sum(dim=1)  
30         user_emb, item_emb = torch.split(  
31             final_embedding, [self.num_users, self.  
32             num_items])  
33         return user_emb, item_emb  
34     def propagate(self, edge_index, edge_weight,  
35         embeddings):  
36         row, col = edge_index  
37         deg = torch.bincount(row, minlength=  
38             embeddings.size(0)).float()  
39         deg_inv_sqrt = deg.pow(-0.5)  
40         deg_inv_sqrt[deg_inv_sqrt == float('inf')] =  
41             0  
42         norm = deg_inv_sqrt[row] * deg_inv_sqrt[col]  
43             * edge_weight  
44         agg = torch.zeros_like(embeddings)  
45         agg.index_add_(0, row, embeddings[col] * norm  
46             .unsqueeze(1))  
47         return agg
```

Ta giải thích từng hàm:

Hàm khởi tạo :

- `super().__init__()`: Gọi hàm khởi tạo của lớp cha (`nn.Module`) để PyTorch biết đây là một mô hình học sâu.
- Các tham số:
 - `num_users`: Tổng số người dùng trong dữ liệu của bạn.
 - `num_items`: Tổng số sản phẩm trong dữ liệu.

- *embedding_dim*: Kích thước vector biểu diễn (embedding) cho mỗi user/item.
- *n_layers*: Số lớp lan truyền thông tin trên đồ thị.
- Tạo một bảng có *num_users* hàng, mỗi hàng là một vector có *embedding_dim* chiều. Mỗi user sẽ được gán một vector riêng biệt (gọi là embedding vector), ví dụ: user 0 có vector $[0.1, -0.2, \dots]$, user 1 có vector khác, v.v. Tương tự, tạo một bảng cho item (sản phẩm), có *num_items* hàng, mỗi hàng là một vector *embedding_dim* chiều.
- Sau đó tiến hành khởi tạo giá trị ban đầu cho embedding. Xavier Uniform là một cách khởi tạo các giá trị ban đầu cho các vector embedding sao cho không quá lớn, không quá nhỏ, giúp mô hình học tốt hơn và ổn định hơn.
- Sử dụng learnable parameter: Sau mỗi lần lan truyền, sẽ có một embedding mới cho mỗi user/item. Learnable parameter sẽ giúp xem là thông tin ở lớp nào đáng tin hơn khi kết hợp lại thành embedding cuối cùng. Khởi tạo giá trị ban đầu cho từng trọng số đều bằng nhau, bằng $\frac{1}{n_layers+1}$. Khi huấn luyện, mô hình sẽ tự động tối ưu alpha này để chọn trọng số kết hợp embedding của từng layer.

```
1  def __init__(self, num_users, num_items,
2      embedding_dim=128, n_layers=2):
3      super().__init__()
4      self.num_users = num_users
5      self.num_items = num_items
6      self.embedding_dim = embedding_dim
7      self.n_layers = n_layers
8      self.user_embedding = nn.Embedding(num_users,
9      embedding_dim)
10     self.item_embedding = nn.Embedding(num_items,
11     embedding_dim)
12     nn.init.xavier_uniform_(self.user_embedding.
13     weight)
14     nn.init.xavier_uniform_(self.item_embedding.
15     weight)
16     self.alpha = nn.Parameter(torch.ones(self.
17     n_layers + 1) / (self.n_layers + 1))
```

Hàm forward định nghĩa toàn bộ quá trình khi mô hình nhận dữ liệu đầu vào và trả ra kết quả. Đầu tiên, ta tiến hành ghép nối các *user_embedding* và *item_embedding* ban đầu thành một tensor lớn *all_embeddings*. Đây là embedding ở Layer 0 và embedding sau mỗi lần lan truyền (layer 1, 2, ...). Lặp qua từng layer của LightGCN, thực hiện gọi *self.propagate* sẽ lan truyền embedding từ node hàng xóm, tạo embedding cho layer tiếp theo (giống công thức LightGCN). Kết quả là embedding mới cho tất cả người dùng, item sau khi lan truyền. Sau mỗi layer, lưu lại embedding mới vào *embeddings_list*.

```
1 def forward(self, edge_index, edge_weight):
2     all_embeddings = torch.cat([self.user_embedding.
3         weight, self.item_embedding.weight])
4     embeddings_list = [all_embeddings]
5     for _ in range(self.n_layers):
6         all_embeddings = self.propagate(edge_index,
            edge_weight, all_embeddings)
7         embeddings_list.append(all_embeddings)
```

Tiếp theo, trong hàm forward ta dùng hàm trên *self.alpha* để ra trọng số layer combination, giúp model tự học xem nên ưu tiên layer nào. Sau đó, kết hợp embedding của các layer (weighted sum) để tìm ra embedding cuối cùng. Cuối cùng tách user, item ra để trả về embedding cuối cho người dùng và item.

```
1 import torch.nn.functional as F
2     weights = F.softmax(self.alpha, dim=0) # [
3         n_layers+1]
4     stacked = torch.stack(embeddings_list, dim=1)
5         # [num_nodes, n_layers+1, emb_dim]
6     final_embedding = (stacked * weights[None, :,
7         None]).sum(dim=1)
8     user_emb, item_emb = torch.split(
9         final_embedding, [self.num_users, self.
10             num_items])
11     return user_emb, item_emb
```

Hàm propagate thực hiện bước lan truyền embedding trên đồ thị user-item nhằm cập nhật embedding cho từng node (user, item) dựa trên thông tin từ node hàng xóm của nó. Ban đầu, hàm này phân tách danh sách các cạnh của đồ thị thành node nguồn (col) và node đích (row), tức là xác định ai truyền embedding cho ai. Sau đó, tính số lượng kết nối mà node đó được nhận (hay còn gọi là bậc). Để đảm bảo giá trị embedding không bị tăng

hoặc giảm quá mức khi lan truyền qua nhiều lớp, mỗi node sẽ được chuẩn hóa bậc bằng cách lấy nghịch đảo căn bậc hai của giá trị degree. Nếu node nào không có kết nối vào (degree bằng 0), hệ số chuẩn hóa này sẽ được đặt thành 0 để tránh lỗi số học. Tiếp theo, với mỗi cạnh trên đồ thị, hàm sẽ tính một hệ số chuẩn hóa dựa trên nghịch đảo căn bậc hai của bậc hai đầu mút và trọng số của cạnh đó (*edge_weight*). Hệ số này giúp cân bằng ảnh hưởng giữa các node có nhiều hay ít kết nối trong quá trình lan truyền thông tin. Từ đây, embedding của các node nguồn sẽ được nhân với hệ số chuẩn hóa tương ứng rồi cộng dồn vào embedding của các node đích.

- Sử dụng `torch.bincount(row, minlength=N)` để đếm số lần mỗi node xuất hiện trong row (tức là số cạnh đi đến node đó $\rightarrow$ bậc vào), còn `minlength=embeddings.size(0)`: đảm bảo độ dài đúng số node (user + item).
- Hệ số chuẩn hóa cho từng cạnh: Sử dụng symmetric normalization – đặc trưng của LightGCN và GCN chuẩn

$$- \text{norm}_{(ij)} = \frac{\text{edge_weight}}{\sqrt{\text{deg}(i)} \times \sqrt{\text{deg}(j)}}$$

```
1 def propagate(self, edge_index, edge_weight,
2   embeddings):
3     row, col = edge_index
4     deg = torch.bincount(row, minlength=embeddings.
5       size(0)).float()
6     deg_inv_sqrt = deg.pow(-0.5)
7     deg_inv_sqrt[deg_inv_sqrt == float('inf')] = 0
8     norm = deg_inv_sqrt[row] * deg_inv_sqrt[col] *
9       edge_weight
10    agg = torch.zeros_like(embeddings)
11    agg.index_add_(0, row, embeddings[col] * norm.
12      unsqueeze(1))
13    return agg
```

5.4.5 Hàm Loss

Hàm này giúp mô hình học được embedding sao cho user sẽ ưu tiên item từng tương tác (positive) hơn item chưa từng tương tác (negative), đồng thời giữ embedding không quá lớn (regularization). Ta sử dụng Bayesian Personalized Ranking là một hàm loss phổ biến trong recommender system

đặc biệt trong dataset mà ta chỉ biết là người dùng có tương tác hay không. Ta kết hợp thêm Regularization để giúp embedding không bị phình quá lớn. Cách hoạt động:

- Đầu tiên, ta tính score giữa user và item positive (item mà đã được tương tác) và score giữa user và item negative (item mà chưa được tương tác). Điểm số này càng cao nghĩa là mô hình càng “tự tin” user sẽ thích item đó.

– Công thức: $score_{u,i} = \mathbf{e}_u^\top \mathbf{e}_i$

- Ta tính loss BPR: Ta lấy hiệu giữa score positive và negative. Nếu score positive lớn hơn negative, loss sẽ nhỏ (tốt). Ngược lại, loss sẽ lớn (xấu). Mục tiêu là model luôn đánh giá item đã tương tác cao hơn item chưa từng tương tác.

– Công thức: $\mathcal{L}_{BPR} = -\frac{1}{N} \sum_{u=1}^N \log \sigma (score_{u,i}^+ - score_{u,i}^-)$

So sánh với code của paper thì ở đây dùng mean() thay vì sum(). Do mean giúp ổn định scale của loss khi thay đổi batch size còn sum thì loss sẽ tăng tỉ lệ thuận với kích thước batch.

- Sau đó ta tính regularization để giữ embedding của user, item không bị quá lớn (tránh overfitting).

– Công thức: $\mathcal{R} = \frac{1}{3N} \sum_{u=1}^N (\|\mathbf{e}_u\|^2 + \|\mathbf{e}_{i^+}\|^2 + \|\mathbf{e}_{i^-}\|^2)$

```
1 def bpr_loss_reg(user_emb, pos_emb, neg_emb,
2   reg_weight=1e-4):
3     pos_scores = (user_emb * pos_emb).sum(dim=1)
4     neg_scores = (user_emb * neg_emb).sum(dim=1)
5     bpr = -torch.log(torch.sigmoid(pos_scores -
6       neg_scores) + 1e-8).mean()
7     reg = (user_emb.norm(2).pow(2) + pos_emb.norm(2).
8       pow(2) + neg_emb.norm(2).pow(2)) / user_emb.
9       size(0) / 3
10    return bpr + reg_weight * reg
```

5.4.6 Hàm sample batch

Hàm này tạo một batch mẫu gồm các bộ ba (user, item positive, item negative) để huấn luyện các mô hình gợi ý, với negative sample (item negative) được lấy dựa trên độ phổ biến (popularity sampling) thay vì chọn ngẫu nhiên hoàn toàn. Trong hệ thống thực, item phổ biến có khả năng gây nhầm lẫn cho model: dễ bị model xếp nhầm vào top recommendation, dù user chưa từng quan tâm. Nếu chỉ chọn negative đều (random hoàn toàn), đa phần sẽ rơi vào các item ít ai quan tâm, quá dễ cho model phân biệt (không mang tính thử thách). Negative là item phổ biến sẽ làm "bài toán khó hơn": model phải học kỹ hơn để không gợi ý các item này nếu user thực sự không thích, dù chúng được nhiều người khác thích.

Cách hoạt động:

- Hàm này được thiết kế để tạo ra các batch dữ liệu phục vụ quá trình huấn luyện mô hình gợi ý, với điểm đặc biệt là negative sample (item negative) được lấy dựa trên độ phổ biến của item thay vì chọn ngẫu nhiên hoàn toàn. Cụ thể, với mỗi batch, hàm sẽ chọn ngẫu nhiên một nhóm user từ danh sách các user trong tập train. Đối với từng user trong nhóm này, hàm tìm tất cả các item mà user đã từng tương tác để làm tập positive. Nếu user chưa từng tương tác với item nào, hàm sẽ chọn ngẫu nhiên cả positive và negative.
- Tuy nhiên, với các user đã có lịch sử tương tác, hàm sẽ chọn ngẫu nhiên một item positive từ tập này, sau đó bắt đầu quá trình chọn item negative. Điểm quan trọng là item negative không được chọn từ toàn bộ tập item một cách đồng đều, mà dựa trên xác suất tỉ lệ với độ phổ biến của từng item (tức là item càng phổ biến thì càng dễ được chọn làm negative). Điều này được đảm bảo nhờ mảng *pop_item_probs* đã được tính toán trước, phản ánh tần suất xuất hiện của các item trong tập train. Hàm sẽ lặp cho đến khi chọn được một item negative mà user chưa từng tương tác để đảm bảo tính hợp lệ của bộ ba mẫu

```
1 unique_users_in_train = train_interactions['user'].  
    unique()  
2 user_pos_items_dict = train_interactions.groupby('user')[  
    'item'].apply(set).to_dict()  
3 item_counts = train_interactions['item'].value_counts()  
    ()
```

```
4 full_item_counts = item_counts.reindex(np.arange(
    num_items), fill_value=1)
5 sampling_probs = np.power(full_item_counts.values,
    0.75)
6 sampling_probs /= np.sum(sampling_probs)
7 item_indices_for_sampling = full_item_counts.index.
    values
8
9 def sample_batch_popularity(batch_size,
    users_to_sample, pos_dict, pop_item_indices,
    pop_item_probs):
10     users = np.random.choice(users_to_sample, size=
        batch_size, replace=True)
11     pos_items, neg_items = [], []
12     for u in users:
13         user_pos_set = pos_dict.get(u, set())
14         if not user_pos_set:
15             pos_items.append(np.random.randint(0,
                num_items))
16             neg_items.append(np.random.randint(0,
                num_items))
17             continue
18         pos_item = np.random.choice(list(user_pos_set
            ))
19         pos_items.append(pos_item)
20         while True:
21             neg_item = np.random.choice(
                pop_item_indices, p=pop_item_probs)
22             if neg_item not in user_pos_set:
23                 neg_items.append(neg_item)
24                 break
25     return torch.LongTensor(users), torch.LongTensor(
        pos_items), torch.LongTensor(neg_items)
```

Trong đó:

- *unique_users_in_train*: Lấy tất cả user xuất hiện trong tập train (dùng để chọn user khi sampling batch).
- *user_pos_items_dict*: Tạo một dictionary cho biết user này đã tương tác với những item nào (tập positive của từng user).

- *item_counts*: Đếm số lần mỗi item xuất hiện trong train (tức là độ phổ biến của mỗi item).
- *full_item_counts*:: Dùng *reindex* để đảm bảo mỗi item đều có một số lần đếm, kể cả item không có mặt trong train (bù = 1).
- *sampling_probs*: Tính xác suất lấy mỗi item theo độ phổ biến:

$$p = \frac{count_i^{0.75}}{\sum_j count_j^{0.75}}$$

Việc cho mũ 0.75 giúp không để item phổ biến chiếm tỷ trọng quá lớn.

- *item_indices_for_sampling*: Lấy ra mảng index của tất cả item (0, 1, ..., *num_items* - 1).

Cách mà hàm *sample_batch_popularity* hoạt động:

- Chọn ngẫu nhiên một batch user từ tập train.
- Với mỗi user:
 - Lấy ra 1 item positive bất kỳ trong các item họ đã tương tác.
 - Lấy 1 item negative chưa từng tương tác, xác suất lấy mỗi item tỉ lệ với độ phổ biến (phổ biến càng dễ được lấy)
- Trả về batch 3-tuple (user, positive, negative) cho huấn luyện.
Lấy 1 item negative chưa từng tương tác, xác suất lấy mỗi item tỉ lệ với độ phổ biến (phổ biến càng dễ được lấy).

5.4.7 Huấn luyện mô hình

Ta sử dụng early stop để tìm kiếm kiến trúc tốt nhất cho dữ liệu. Ở đây ta cho mặc định số chiều là 256, learning rate là 0.0005, dùng *HR@20* và *NDCG@20* để đánh giá mô hình trên tập validation từ đó so sánh layer bằng 2 và 3.

```
1 architectures_to_test = [2,3]
2 results = {}
3 best_overall_model_state = None
4 winning_architecture_hr = -1
5 best_n_layers = 0
```

```
6     best_epoch = 0
7     training_history = {}
8     K = 20
9     embedding_dim = 256
```

Giải thích tên biến:

- *architectures_to_test*: Danh sách số lớp muốn thử nghiệm (ở đây là 2 và 3)
- *results*: Dict để lưu kết quả tốt nhất (HR và epoch) của mỗi kiến trúc.
- *best_overall_model_state*, *winning_architecture_hr*, *best_n_layers*, *best_epoch*: Biến toàn cục để theo dõi kiến trúc và trạng thái model tốt nhất trên tất cả các thử nghiệm.
- *training_history*: Dict để lưu lịch sử loss, HR, NDCG của từng epoch cho mỗi *n_layers*, phục vụ cho việc phân tích sau.
- *K*: Giá trị K trong *HR@K*, *NDCG@K*.
- *embedding_dim*: Kích thước embedding của user/item.

```
1     for n_layers in architectures_to_test:
2         print(f"===== TESTING ARCHITECTURE:
3             n_layers = {n_layers} =====")
4         training_history[n_layers] = {
5             'epochs': [],
6             'losses': [],
7             'hr_scores': [],
8             'ndcg_scores': []
9         }
10
11         model = LightGCN(num_users, num_items,
12                           embedding_dim, n_layers=n_layers)
13         optimizer = optim.Adam(model.parameters(), lr
14                                 =0.0005)
15
16         patience = 7
17         wait = 0
18         best_hr_for_this_arch = 0
19         best_model_state_for_this_arch = None
20         best_epoch_for_this_arch = 0
```

```
18
19     epochs = 30
20     batch_size = 1024
21     num_batches_per_epoch = len(train_interactions)
22         // batch_size if len(train_interactions) >
23         batch_size else 1
24
25     for epoch in range(epochs):
26         model.train()
27         total_loss = 0
28         for _ in range(num_batches_per_epoch):
29             users, pos_items, neg_items =
30                 sample_batch_popularity(batch_size,
31                     unique_users_in_train,
32                     user_pos_items_dict,
33                     item_indices_for_sampling,
34                     sampling_probs)
35             user_emb, item_emb = model(edge_index,
36                 edge_weight)
37             u_emb, pos_emb, neg_emb = user_emb[users
38                 ], item_emb[pos_items], item_emb[
39                 neg_items]
40
41             optimizer.zero_grad()
42             loss = bpr_loss_reg(u_emb, pos_emb,
43                 neg_emb, reg_weight=1e-4)
44             loss.backward()
45             optimizer.step()
46             total_loss += loss.item()
47
48         avg_loss = total_loss / num_batches_per_epoch
49         if num_batches_per_epoch > 0 else 0
50         print(f"Epoch {epoch+1}/{epochs}, Loss: {
51             avg_loss:.4f}")
52
53         print("Evaluate on validation data...")
54         hr, ndcg = evaluate_lightgcn(model,
55             train_interactions, val_interactions,
56             edge_index, edge_weight, K=K)
57
58         training_history[n_layers]['epochs'].append(
```

```
epoch + 1)
44     training_history[n_layers]['losses'].append(
        avg_loss)
45     training_history[n_layers]['hr_scores'].
        append(hr)
46     training_history[n_layers]['ndcg_scores'].
        append(ndcg)
47     if hr > best_hr_for_this_arch:
48         best_hr_for_this_arch = hr
49         best_epoch_for_this_arch = epoch + 1
50         wait = 0
51         best_model_state_for_this_arch = model.
            state_dict()
52         print(f"--> Validation HR improves: {
            best_hr_for_this_arch:.4f} at epoch {
            best_epoch_for_this_arch}. Save model.
            ")
53     else:
54         wait += 1
55         if wait >= patience:
56             print(f"--> Validation HR does not
                improve in {patience} epochs.
                EARLY STOPPING!")
57             break
```

Với mỗi kiến trúc, chương trình sẽ khởi tạo một mô hình mới, sử dụng optimizer Adam và thiết lập cơ chế "early stopping" để dừng huấn luyện sớm nếu mô hình không cải thiện hiệu suất trong một số epoch liên tiếp (ở đây là 7 epoch - epoch là học qua toàn bộ dữ liệu một lần). Ở mỗi epoch, chúng ta không thể huấn luyện một lần hết tất cả dòng mà phải chia ra thành những batch nhỏ. Ở mỗi batch nhỏ này, chúng ta thực hiện sampling người dùng, item positive và item negative, sau đó tính embedding của người dùng và item bằng cách truyền qua mô hình LightGCN. Các embedding này được sử dụng để tính toán hàm mất mát BPR (Bayesian Personalized Ranking Loss), từ đó mô hình học cách ưu tiên item dương hơn item âm trong quá trình gợi ý. Sau khi tính toán gradient, mô hình được cập nhật thông qua bước tối ưu (loss.backward() → tính gradient, optimizer.step() → cập nhật trọng số).

Kết thúc mỗi epoch, mô hình được đánh giá trên tập validation bằng hai chỉ số: HR@K và NDCG@K. Các giá trị loss, HR và NDCG của từng epoch

được lưu lại để theo dõi tiến trình huấn luyện. Nếu mô hình đạt được HR cao hơn trước đó, trạng thái mô hình hiện tại sẽ được lưu lại làm "mô hình tốt nhất" cho kiến trúc đó. Ngược lại, nếu HR không cải thiện liên tiếp nhiều epoch, quá trình huấn luyện sẽ dừng sớm nhằm tránh overfitting và tiết kiệm tài nguyên.

```
1      results[n_layers] = (best_hr_for_this_arch ,
2                          best_epoch_for_this_arch)
3      print(f"The best result with n_layers = {n_layers}
4            ): HR@{K} = {best_hr_for_this_arch:.4f} t i
5            epoch {best_epoch_for_this_arch}")
6      if best_hr_for_this_arch >
7          winning_architecture_hr:
8          winning_architecture_hr =
9              best_hr_for_this_arch
10         best_overall_model_state =
11             best_model_state_for_this_arch
12         best_n_layers = n_layers
13         best_epoch = best_epoch_for_this_arch
```

Sau khi ta tìm được kiến trúc tốt nhất, ta lưu kết quả này lại. Sau đó, ta tiến hành Khởi tạo model với kiến trúc chiến thắng. Sau đó, đánh giá trên tập test.

```
1      final_model = LightGCN(num_users , num_items ,
2                              embedding_dim , n_layers=best_n_layers)
3      if best_overall_model_state:
4          final_model.load_state_dict(
5              best_overall_model_state)
6          model = final_model
7      else:
8          print("Error")
9          final_model = None
10     if final_model:
11         evaluate_lightgcn(final_model , train_interactions
12                           , test_interactions , edge_index , edge_weight ,
13                           K=K)
```

5.4.8 Hàm đánh giá LightGCN

Hàm này đánh giá mô hình gợi ý bằng cách kiểm tra với từng user, model có gợi ý đúng item test (item mới) của user trong top K gợi ý không, và vị

trí xuất hiện của item đó. Ta sử dụng hai chỉ số để đánh giá là $HR@K$ và $NDCG@K$.

$HR@K$ là tỉ lệ số lần item đúng (item test – item user thực sự quan tâm) nằm trong top K gợi ý của mô hình. Nếu test item xuất hiện trong top K gợi ý: tính là 1 (hit). Nếu không xuất hiện: tính là 0 (miss).

$NDCG@K$ là chỉ số đánh giá chất lượng thứ hạng: Không chỉ xem test item có trong top K không, mà còn xuất hiện ở vị trí nào trong top K. Ví dụ: Nếu test item ở top 1: $NDCG = 1$, ở top 2 thì $NDCG = 1 / \log_2(3) \approx 0.6309$, ở top 3: $NDCG = 1 / \log_2(4) = 0.5$.

```
1  def evaluate_lightgcn(model, train_interactions,
2      test_interactions, edge_index, edge_weight, K=20):
3      model.eval()
4      user_emb, item_emb = model(edge_index,
5          edge_weight)
6      user_emb = user_emb.cpu().detach().numpy()
7      item_emb = item_emb.cpu().detach().numpy()
8      hits, ndcgs = [], []
9      user2train_items = train_interactions.groupby('
10         user')['item'].apply(set).to_dict()
11      user2test_item = test_interactions.set_index('
12         user')['item'].to_dict()
13      for user in user2test_item:
14         test_item = user2test_item[user]
15         train_items = user2train_items.get(user, set
16             ())
17         scores = np.dot(user_emb[user], item_emb.T)
18         scores[list(train_items)] = -np.inf
19         top_k = np.argpartition(scores, -K)[-K:]
20         top_k = top_k[np.argsort(-scores[top_k])]
21         hit = int(test_item in top_k)
22         hits.append(hit)
23         if test_item in top_k:
24             rank = np.where(top_k == test_item)[0][0]
25             + 1
26             ndcg = 1 / np.log2(rank + 1)
27         else:
28             ndcg = 0
29         ndcgs.append(ndcg)
30     hr = np.mean(hits)
31     ndcg = np.mean(ndcgs)
```

```
26     print(f"HR@{K}: {hr:.4f}, NDCG@{K}: {ndcg:.4f}")
27     return hr, ndcg
```

5.4.9 Hàm recommend

Mục đích của hàm này là để đánh giá hiệu quả của mô hình gợi ý theo tiêu chí $HR@20$ – tức là chỉ cần item đúng xuất hiện trong top 20 là được tính là đúng.

```
1     def recommend_top_k(model, user_id, k):
2         model.eval()
3         with torch.no_grad():
4             user_emb, item_emb = model(edge_index,
5                                       edge_weight)
6             u_idx = user_encoder.transform([user_id])[0]
7             scores = (user_emb[u_idx] @ item_emb.t()).cpu()
8             .numpy()
9             top_k = np.argsort(-scores)[:k]
10            return item_encoder.inverse_transform(top_k)
11
12    user_to_check = test_interactions['visitorid'].iloc
13    [5]
14    user_internal_id = user_encoder.transform([
15        user_to_check])[0]
16    ground_truth_item = test_interactions[
17        test_interactions['visitorid'] == user_to_check]['
18        itemid'].iloc[0]
19    recommendations = recommend_top_k(final_model,
20        user_to_check, k=20)
21    if ground_truth_item in recommendations:
22        print("\n==> Success!")
23    else:
24        print("\n==> Fail ")
```

Cách hoạt động của hàm:

- Đưa model về chế độ đánh giá (không dùng dropout, batchnorm...) để kết quả ổn định.
- Lấy embedding của toàn bộ user và item từ mô hình đã học. Chuyển $user_id$ thành chỉ số nội bộ đã mã hóa (0, 1, 2, ...).

- Tính điểm (score) giữa user này với tất cả các item (dùng tích vô hướng giữa embedding user và embedding item) rồi sắp xếp lại theo score giảm dần, lấy K item cao nhất.

5.5 Hiện thực LightGCN kết hợp thêm feature

Phương pháp này đại diện cho một hướng tiếp cận hybrid hiệu quả, nơi thông tin side information được bổ sung trực tiếp vào kiến trúc LightGCN mà không làm thay đổi cấu trúc đồ thị gốc. Cụ thể, mỗi item trong hệ thống, bên cạnh embedding học được từ tương tác user-item, còn được làm giàu thêm bằng embedding của feature mà item đó thuộc về. Việc cộng embedding feature vào embedding item giúp mô hình khai thác được các đặc trưng phân loại sản phẩm – một yếu tố quan trọng trong thương mại điện tử – đồng thời duy trì được sự đơn giản, “nhẹ” vốn có của LightGCN, bởi các phép lan truyền graph convolution vẫn chỉ diễn ra trong không gian User-Item truyền thống. Cách tiếp cận này giúp mô hình xử lý tốt hơn các trường hợp cold-start hoặc long-tail, khi mà chỉ dựa vào lịch sử tương tác là chưa đủ. Việc tận dụng embedding category như một auxiliary feature đã chứng minh được hiệu quả rõ rệt trong việc nâng cao chất lượng cá nhân hóa và khả năng tổng quát của hệ thống gợi ý mà không cần phải mở rộng graph hoặc áp dụng các phép biến đổi phức tạp.

Ta sử dụng lại các hàm chia train/test/validation, Loss, sample batch, đánh giá, recommend phía trên.

5.5.1 Tiền xử lý dữ liệu *item_properties*

Tiền xử lý dữ liệu "events.csv" giống như trên.

Ta tiến hành tiền xử lý cho hai dataset là "*item_properties_part1.csv*" và "*item_properties_part2.csv*". Đầu tiên, ta load dữ liệu vào

```
1 props1 = pd.read_csv('item_properties_part1.csv')
2 props2 = pd.read_csv('item_properties_part2.csv')
```

Sau đó, ta kiểm tra thông tin của hai file

```
1 def check_info_prop(data):
2     print('The size of the dataset:', data.shape)
3     print('The itemid unique:', data['itemid'].nunique())
```

```
4     print('The property unique:', data['property'].  
      unique())  
5     print(data['property'].value_counts())  
6     check_info_prop(props1)  
7     check_info_prop(props2)
```

Kết quả:

```
The size of the dataset: (10999999, 4)  
The itemid unique: 417053  
The property unique: 1097  
property  
888      1629817  
790      970800  
available 817387  
categoryid 426305  
6         343207  
...  
598      1  
244      1  
424      1  
494      1  
712      1
```

Figure 29: Thông tin props1

```
The size of the dataset: (9275903, 4)  
The itemid unique: 417053  
The property unique: 1094  
property  
888      1370581  
790      819716  
available 686252  
categoryid 361909  
6         288264  
...  
494      1  
57       1  
1091     1  
522      1  
769      1  
Name: count, Length: 1094, dtype: int64
```

Figure 30: Thông tin props2

Tiếp theo, ta kiểm tra xem top 10 property xuất hiện nhiều nhất

```
1  def top10_high(data,x):
2      if x=='property':
3          counts = data[x].value_counts()
4          top10_high_counts = counts.head(10)
5          plt.figure(figsize=(10, 6))
6          bars = plt.bar(top10_high_counts.index.astype(str), top10_high_counts.values, color='#c94169')
7          plt.xlabel('Property ID')
8          plt.ylabel('Number of Occurrences')
9          plt.title('Top 10 Most Frequent Properties')
10         plt.xticks(rotation=90)
11         plt.tight_layout()
12
13
14         for x in bars:
15             yval = x.get_height()
16             plt.text(x.get_x() + x.get_width()/2, yval,
17                     round(yval, 2), va='bottom', ha='center')
18             # Display value above bar
19
20         plt.show()
21     else:
22         print('Error: This function is designed for
23               property data.')
24 top10_high(props1,'property')
25 top10_high(props2,'property')
```

Kết quả:

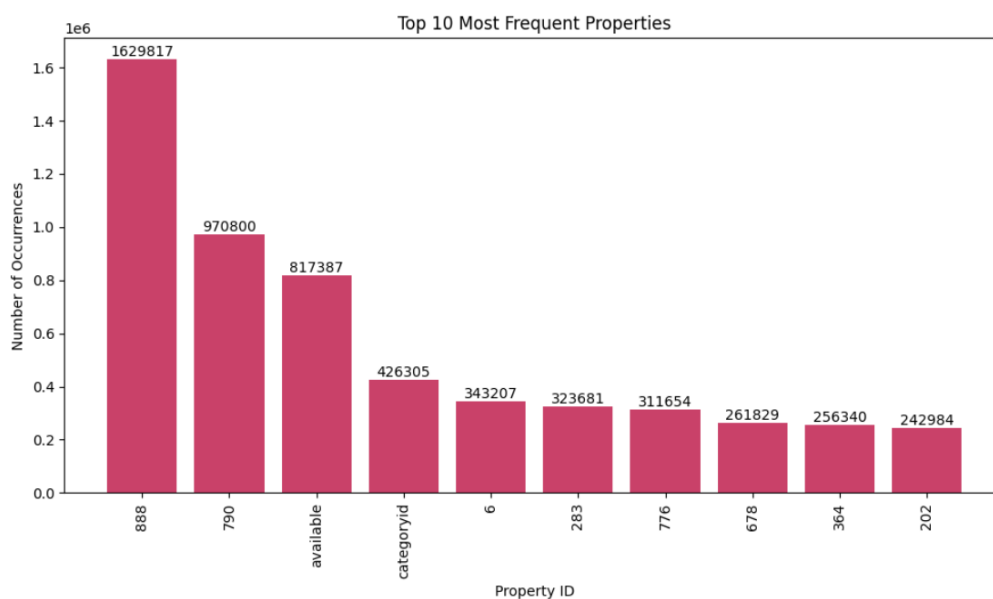


Figure 31: Biểu đồ top 10 properties xuất hiện nhiều nhất ở props1

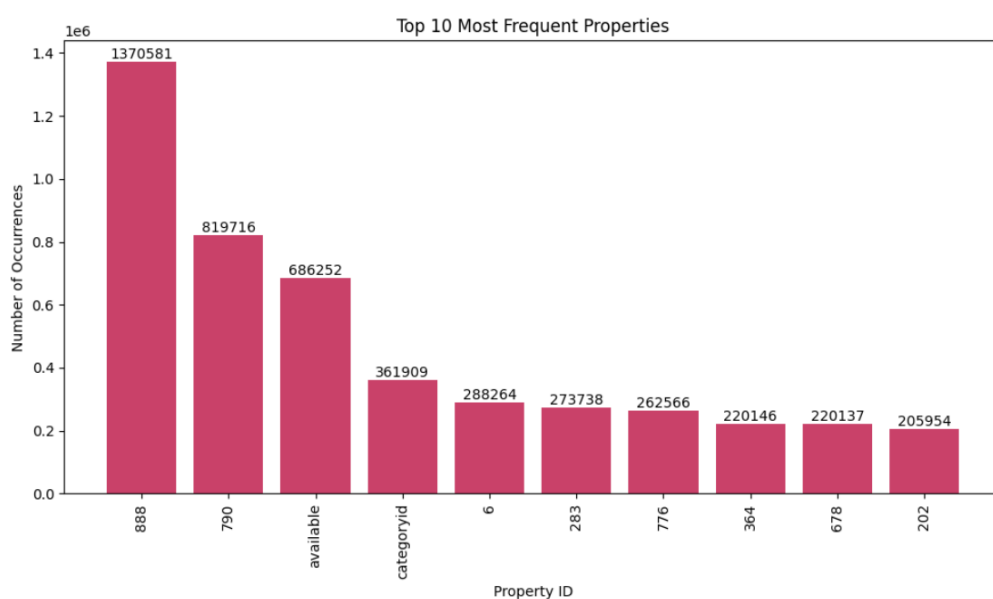


Figure 32: Biểu đồ top 10 properties xuất hiện nhiều nhất ở props2

Nhận xét: Trong top 10 xuất hiện nhiều nhất ở cả hai props1 và props2, thì thuộc tính "available" và "categoryid" là thuộc tính không bị mã hóa, mang ý nghĩa rõ ràng. Trong khi đó, thuộc tính khác như "888" hay "790"

dù có tần suất áp đảo nhưng không thể xác định ý nghĩa cụ thể nếu không có ảnh xạ ngược.

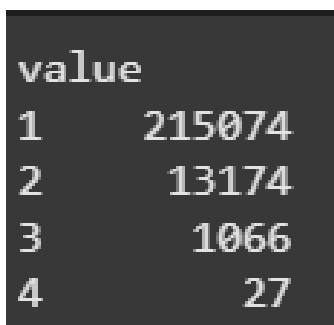
"available" về mặt ý nghĩa thì chỉ đánh dấu xem là mặt hàng đó còn hàng hay không. Nó không giúp phân loại rõ ràng giữa các item. Vì nó không phản ánh được bản chất item. Mô hình khó học được thông tin sâu từ chỉ từ giá trị của "available".

Còn "categoryid" thì mang ý nghĩa phân loại rõ ràng đó là phân loại nhóm hàng. Đây là yếu tố quan trọng trong hành vi người dùng (người dùng thường quan tâm đến nhóm sản phẩm cụ thể). Do đó, chúng ta sẽ chọn "categoryid" kết hợp mô hình LightGCN.

Sau đó, ta kiểm tra xem mỗi itemid có property là "categoryid" có bao nhiêu giá trị riêng biệt.

```
1 category_properties = props1[props1['property'] == 'categoryid']
2 categoryid_counts_per_item = category_properties.groupby('itemid')['value'].nunique()
3 distribution = categoryid_counts_per_item.value_counts()
4 print(distribution)
```

Kết quả:



value	
1	215074
2	13174
3	1066
4	27

Figure 33: Tần suất các giá trị riêng biệt mà mỗi categoryid sở hữu

```
1 bars = plt.bar(distribution.index.astype(str), distribution.values, color='#c94169')
2 plt.xlabel('Value')
3 plt.ylabel('Number of itemids ')
4 plt.title('The distribution of the number of unique value of categoryid')
```



```
5 plt.yscale('log')  
6 plt.tight_layout()
```

Kết quả:

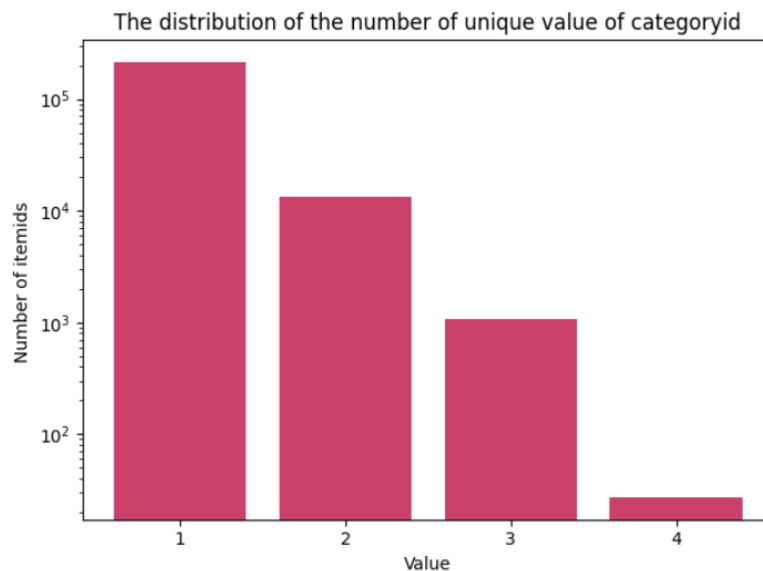


Figure 34: Biểu đồ tần suất các giá trị riêng biệt mà mỗi categoryid sở hữu

Nhận xét: Ta nhận thấy phần lớn dữ liệu chỉ có một giá trị category duy nhất. Nhưng dữ liệu có từ hai trở lên cũng rất nhiều. Ta cần chọn một giá trị category cho item có nhiều giá trị category. Ta tiến hành gộp dữ liệu props1 và props2 lại. Với những itemid có nhiều giá trị category thì ta chọn giá trị gần nhất theo timestamp. Ta nối dữ liệu category vào data event. Những item không có category thì ta điền giá trị -1.

```
1 from sklearn.preprocessing import LabelEncoder  
2 print("Start processing item properties...")  
3 item_props = pd.concat([props1, props2], ignore_index  
4                        =True)  
5 print("Finish reading and joining 2 file properties."  
6      )  
7  
8 if not item_props.empty:  
9     cat_df = item_props[item_props['property'] == '  
10                    categoryid'].copy()
```

```
8         latest_cat_df = cat_df.sort_values('timestamp').  
          drop_duplicates('itemid', keep='last')  
9         latest_cat_df = latest_cat_df[['itemid', 'value']  
          ].rename(columns={'value': 'categoryid'})  
10  
11         final_events_with_cat = pd.merge(filtered_events,  
          latest_cat_df, on='itemid', how='left')  
12     else:  
13         final_events_with_cat = filtered_events.copy()  
14         final_events_with_cat['categoryid'] = -1  
15     print("The number of null values: ",  
          final_events_with_cat['categoryid'].isnull().sum()  
          )  
16     final_events_with_cat['categoryid'].fillna(-1,  
          inplace=True)  
17     print("Finish joining.")
```

5.5.2 Chuẩn bị dữ liệu đầu vào

Đầu tiên, ta tiến hành chuyển dữ liệu user, item, weight sang dạng tensor phù hợp. Từ hai tensor chỉ số user và item, mã tạo ra *edge_index*. sau đó, thực hiện ánh xạ từ item sang *category_idx*. Việc này được thực hiện bằng lọc các cặp (item, *category_idx*) không trùng lặp và sắp xếp theo chỉ số của user. Kết quả được chuyển thành, giúp mô hình thuộc danh mục nào. Tất cả những tensor sau cùng sẽ được chuyển sang thiết bị phù hợp để chuẩn bị đưa vào mô hình.

```
1     user_indices = torch.tensor(train_interactions['user']  
2     ].values, dtype=torch.long)  
3     item_indices = torch.tensor(train_interactions['item']  
4     ].values, dtype=torch.long)  
5     edge_weight = torch.tensor(train_interactions['weight']  
6     ].values, dtype=torch.float)  
7  
8     edge_index = torch.stack([user_indices, item_indices  
9     ], dim=0)  
10  
11     item_to_cat_map_df = interactions[['item', '  
12     category_idx']].drop_duplicates('item').  
13     sort_values('item')
```

```
8 itemid2cat_tensor = torch.tensor(item_to_cat_map_df['  
    category_idx'].values, dtype=torch.long)  
9  
10 device = torch.device("cuda" if torch.cuda.  
    is_available() else "cpu")  
11 edge_index = edge_index.to(device)  
12 edge_weight = edge_weight.to(device)  
13 itemid2cat_tensor = itemid2cat_tensor.to(device)
```

5.5.3 LightGCN kết hợp feature

Tương tự như ở trên nhưng khác biệt ở chỗ mỗi item được cộng thêm vector đại diện cho danh mục của nó, từ embedding *self.category_embedding*. Lúc này, thay vì chạy hai hệ thống riêng biệt rồi trộn kết quả, mô hình Category-enhanced LightGCN tích hợp thông tin category vào ngay trong quá trình học của LightGCN.

```
1 import torch  
2 import torch.nn as nn  
3 import torch.nn.functional as F  
4  
5 class LightGCN_with_Category(nn.Module):  
6     def __init__(self, num_users, num_items,  
7         num_categories, embedding_dim=128, n_layers=2)  
8         :  
9         super().__init__()  
10        self.num_users = num_users  
11        self.num_items = num_items  
12        self.embedding_dim = embedding_dim  
13        self.n_layers = n_layers  
14  
15        self.embedding = nn.Embedding(num_users +  
            num_items, embedding_dim)  
16        self.category_embedding = nn.Embedding(  
            num_categories, embedding_dim)  
17        self.alpha = nn.Parameter(torch.ones(self.  
            n_layers + 1) / (self.n_layers + 1))  
18  
19        self.reset_parameters()  
20  
21    def reset_parameters(self):
```

```
20         nn.init.xavier_uniform_(self.embedding.weight
21         )
22         nn.init.xavier_uniform_(self.
23             category_embedding.weight)
24
25     def propagate(self, edge_index, edge_norm,
26         embeddings):
27         row, col = edge_index
28         agg = torch.zeros_like(embeddings)
29         agg.index_add_(0, row, embeddings[col] *
30             edge_norm.unsqueeze(1))
31         return agg
32
33     def get_embedding(self, edge_index, edge_weight,
34         itemid2cat_tensor):
35         all_initial_embeddings = self.embedding.
36             weight.clone()
37         cat_embeds = self.category_embedding(
38             itemid2cat_tensor)
39         all_initial_embeddings[self.num_users:] +=
40             cat_embeds
41
42         row, col = edge_index
43         item_indices_shifted = col + self.num_users
44         full_edge_index = torch.cat([torch.stack([row
45             , item_indices_shifted]), torch.stack([
46             item_indices_shifted, row])], dim=1)
47
48         full_row, full_col = full_edge_index
49         deg = torch.bincount(full_row, minlength=self
50             .num_users + self.num_items).float()
51         deg_inv_sqrt = deg.pow(-0.5)
52         deg_inv_sqrt[deg_inv_sqrt == float('inf')] =
53             0
54         edge_norm = deg_inv_sqrt[full_row] *
55             deg_inv_sqrt[full_col]
56
57         if edge_weight is not None:
58             full_edge_weight = torch.cat([edge_weight
59                 , edge_weight])
60             edge_norm = edge_norm * full_edge_weight
```

```
47         embeddings_list = [all_initial_embeddings]
48         current_embeddings = all_initial_embeddings
49         for _ in range(self.n_layers):
50             current_embeddings = self.propagate(
51                 full_edge_index, edge_norm,
52                 current_embeddings)
53             embeddings_list.append(current_embeddings)
54
55         stacked_embeddings = torch.stack(
56             embeddings_list, dim=1)
57         layer_weights = F.softmax(self.alpha, dim=0)
58         final_embeddings = (stacked_embeddings *
59                             layer_weights[None, :, None]).sum(dim=1)
60         users_emb, items_emb = torch.split(
61             final_embeddings, [self.num_users, self.
62                               num_items])
63         return users_emb, items_emb
64
65     def forward(self, edge_index, edge_weight,
66                 itemid2cat_tensor):
67         return self.get_embedding(edge_index,
68                                   edge_weight, itemid2cat_tensor)
```

5.6 Thử nghiệm

Đây là đồ thị so sánh $HR@20$, $NDCG@20$ và $Loss$ của layer bằng 2 và 3 theo từng epoch của quá trình huấn luyện mô hình LightGCN

- Ta nhận thấy từ trong những epoch đầu thì layer bằng 3 cho kết quả $HR@20$ vượt trội hơn nhiều so với layer=2. Nhưng sau đó, layer bằng 2 kết quả cho ra kết quả vượt trội hơn layer bằng 3. Khoảng sau đó thì layer bằng 3 có tăng lên nhưng vẫn thấp hơn layer bằng 2. Kết quả cuối cùng cho thấy rằng layer bằng 2 cho kết quả tốt nhất là $HR@20 = 0.0651$ tại epoch 9. Còn layer bằng 3 thì kết quả $HR@20$ tốt nhất là 0.0632 tại epoch 11.
- Tiếp theo, ta so sánh $NDCG@20$ của layer bằng 2 và 3. Tương tự như $HR@20$, ban đầu layer bằng 3 cho kết quả vượt trội hơn nhưng sau

epoch 3 thì kết quả của layer bằng 2 cao hơn hẳn layer bằng 3 cho đến khi layer bằng 2 và 3 bị dừng cho early stop.

- Loss của cả hai layer đều giảm qua mỗi epoch nhưng layer bằng ba kết quả loss giảm mạnh hơn.

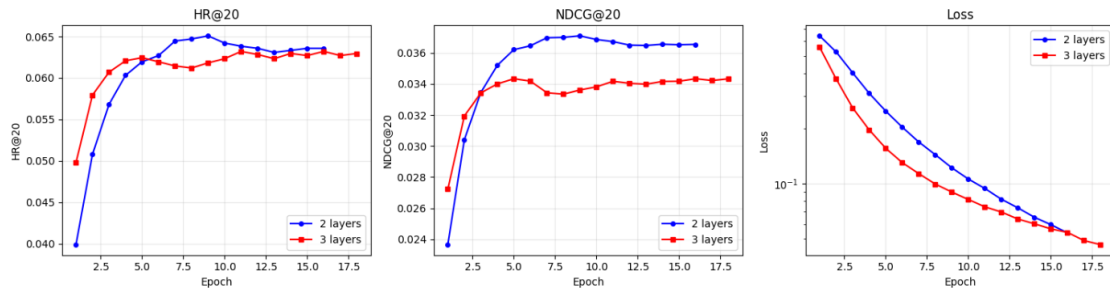


Figure 35: Biểu đồ so sánh HR@20, N DCG@20 và Loss của layer bằng 2 và 3 theo từng epoch của LightGCN thuần

Đây là đồ thị so sánh $HR@20$, $NDCG@20$ và $Loss$ của layer bằng 2 và 3 theo từng epoch của quá trình huấn luyện mô hình LightGCN kết hợp thêm thuộc tính "categoryid".

- Xét về $HR@20$ và $NDCG@20$ thì layer bằng 3 cho kết quả vượt trội hơn hẳn so với layer bằng 2.
- Xét về Loss thì cả hai layer đều giảm qua mỗi epoch, nhưng layer bằng 3 giảm mạnh hơn.

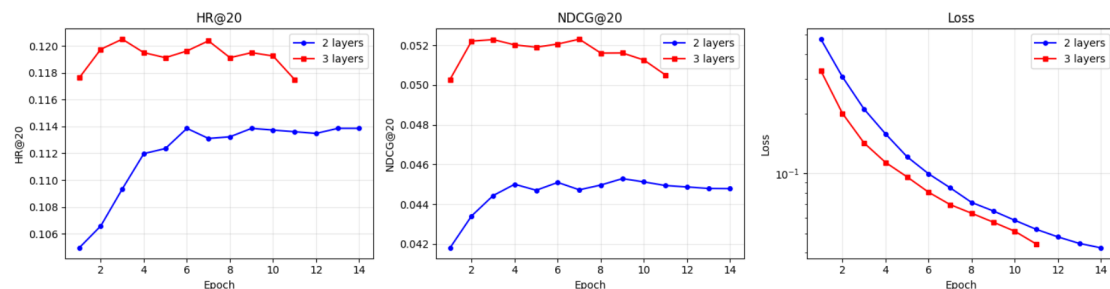


Figure 36: Biểu đồ so sánh HR@20, N DCG@20 và Loss của layer bằng 2 và 3 theo từng epoch của LightGCN kết hợp thêm "Category"

Đây là biểu đồ so sánh hiệu suất của mô hình LightGCN thuần và LightGCN kết hợp với category. Ta có nhận thấy rõ ràng, kết quả của mô hình có kết hợp Category tốt hơn rất nhiều LightGCN. Mô hình Category-enhanced LightGCN cho kết quả $HR@20$ tốt hơn LightGCN gấp 2.6 lần, $NDCG@20$ gấp 2 lần. Điều này chứng tỏ, việc kết hợp category đã giúp mô hình học tốt hơn các mối liên hệ tiềm ẩn giữa các item cùng loại. Chất lượng recommendation được nâng cao, dù dữ liệu có hiện tượng cold-start và dữ liệu thưa.

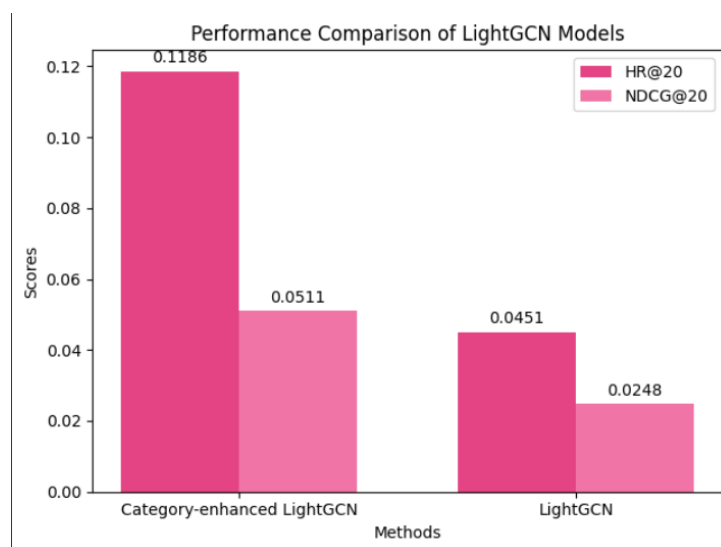


Figure 37: Biểu đồ so sánh hiệu suất của các mô hình LightGCN

6 Tổng kết

LightGCN là phương pháp đơn giản, dễ triển khai, tận dụng cốt lõi mối quan hệ giữa user-item, bỏ đi feature transformation và nonlinear activation để đưa ra những dự đoán sản phẩm cho người dùng. Với dữ liệu RetailRocket thì rất thưa, có hiện tượng cold-start, nên để mô hình LightGCN thuần thì kết quả đạt được là 0.0451 là tương đối ổn. Việc kết hợp embedding category vào LightGCN là một hướng hybrid đơn giản nhưng mang lại hiệu quả lớn, giải quyết được vấn đề cold-start và độ thưa cao.



7 Tài liệu tham khảo

1. [A Gentle Introduction to Graph Neural Networks](#)
2. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation - Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, Meng Wang
3. Recommender Systems - Charu C. Aggarwal.